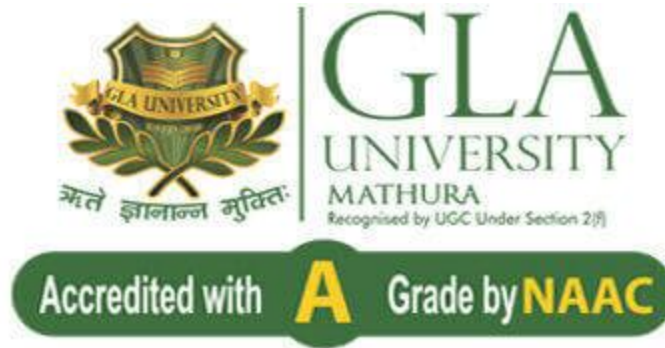




COMPUTER ORGANIZATION AND ARCHITECTURE (BCAC1008)

Course:BCA II Year III Semester

Anuj Kumar

Assistant Professor

anujkumar.gla@gla.ac.in

9997189728

Computer Organization

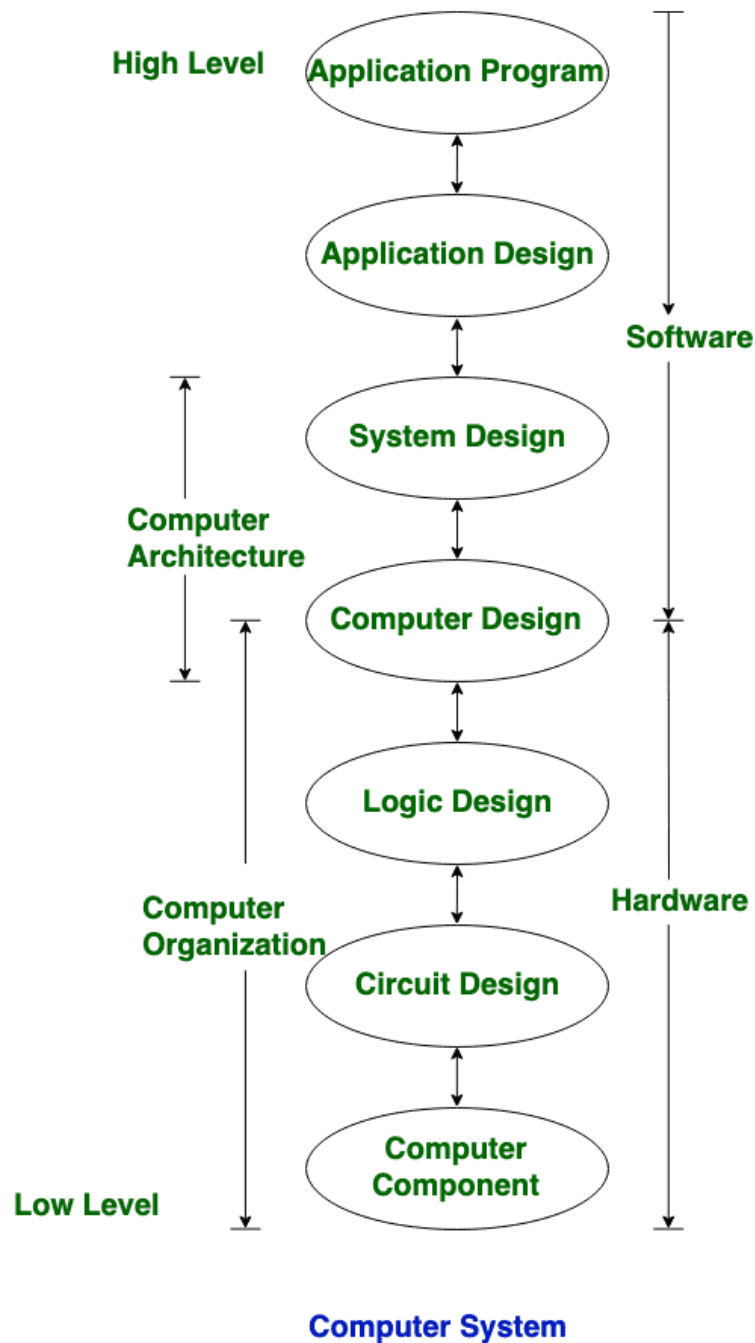
- Computer Organization refers to the operational units and their interconnections.
- Computer Organization is concerned with the way the hardware components operate and the way they are connected together to form the computer system.
- Computer Organization explains how a computer works
- <https://www.youtube.com/watch?v=DsK35f8wyUw>

- Deals with Low Level Design Issues .
- Organizational attributes include those hardware details transparent to the programmer, such as control signals, interfaces between computer and peripherals, and the memory technology used.
- Physical components like circuits with Adder , Signals, Peripheral etc

Computer Architecture

- Computer Architecture is describing the capabilities and programming model of a computer but not a particular implementation .
- It includes the instruction formats, the instruction sets, and techniques for addressing memory.
- Architectural design of a computer system is concerned with the various functional modules, such as processors and memories, and structuring them together into computer system.

- Deals with High Level Design Issues.
- For designing a computer its architecture is fixed first.
- It describes ‘What the system does.
- It co ordinate between hardware and software of the system
- Example Instruction Set, Addressing Modes, Data Type



Difference CA and CO example

- Intel and AMD makes X86 CPU where X86 refers to the computer architecture used.
- (One architecture X86)
- But two different computer organization (INTEL and AMD Flavours)

There are two basic types of architecture

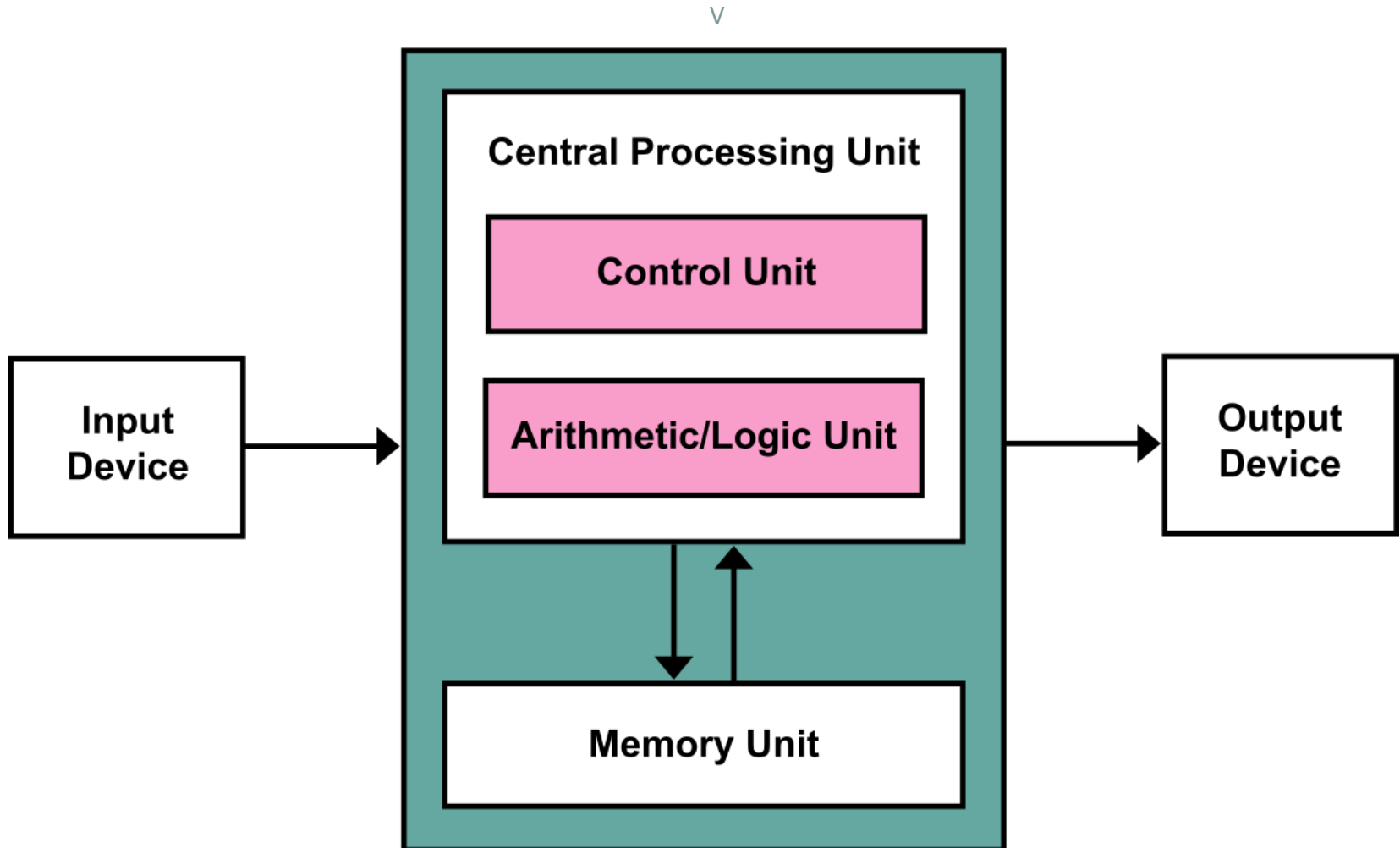
- **Von Neumann Architecture**
- **Harvard Architecture**

Von Neumann Architecture(Stored Memory Program)

- In the Von Neumann Architecture, Program and data are stored in the same memory and managed by the same information handling subsystem. Example : Desktop Personal Computer.
- The Von Neumann Architecture has a single storage structure to hold both instructions and data.
- The CPU can be either reading an instruction or reading / writing data from / to the memory because instructions and data use the same BUS system.

- According to Von Neumann Architecture structure of computer system as composed of the following components:-
 - Control Unit
 - Arithmetic and Logic Unit
 - Main Memory
 - Registers
 - Inputs/Outputs Unit

Von Neumann Architecture(Diagram)



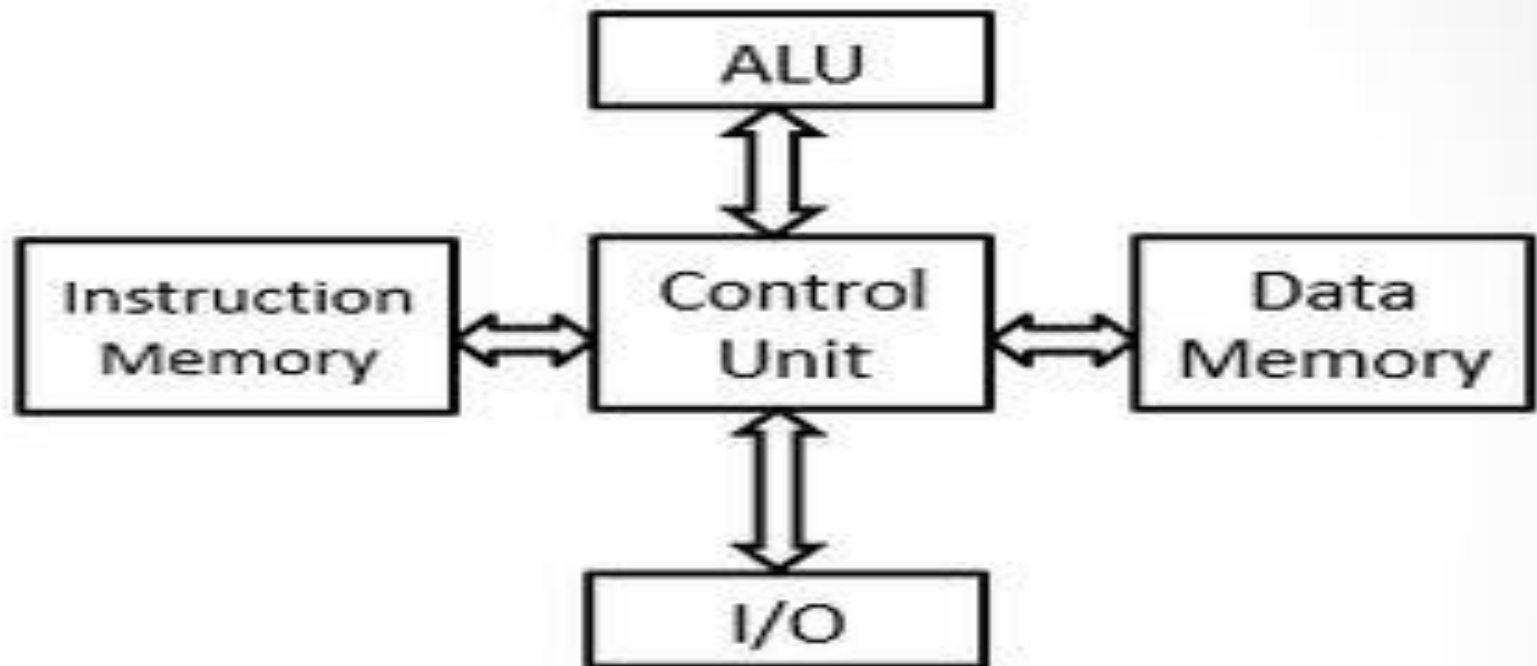
Harvard Architecture

- The Harvard Architecture has a physically separated storage and signal pathways for instructions and data.
- In the Harvard Architectural Computer, the CPU can read both an instruction and data from memory at the same time, leading to double the memory bandwidth.

- In the Harvard architecture, program and data are stored and handled by different subsystems.
- **Example:** - DSP (Digital signal Processor) based computer systems.
- One example of Harvard Architecture is the early computer Mainframe systems where instructions are stored in one programming media such as punch cards and data are stored in the other [programming media such as Tapes.

- A Microcontroller with Harvard Architecture can both read an instruction and perform a data memory access at the same time, even without a cache.
- A Harvard Architecture microcontroller can thus be faster for a given circuit complexity because instruction fetches and data access do not contend for a single memory pathway.

HARDVARD ARCHITECTURE



Harvard Model

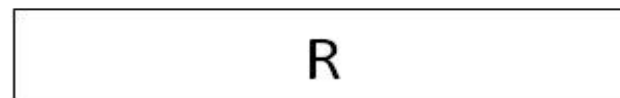
Registers

- Computer registers are designated by capital letters to denote the function of the register.
- **For example:**
- **MAR** -Memory Address Register
- **PC** -Program Counter
- **IR** - Instruction Register
- **R1** - Processor Register.
- **ACC**-Acumulator Register

- In this register, if the register is of n -bit then the numbering is started in sequence from 0 to $n-1$ starting from 0 in the right most position and increasing the number toward leftmost side.
- The representation of register in block diagram form the most common way to represent a register is by a rectangular box with the name of register inside.

Fig. 4-1 a. Register representation by a rectangular box. The individual bits can be distinguished in **Fig .4-1 b.**

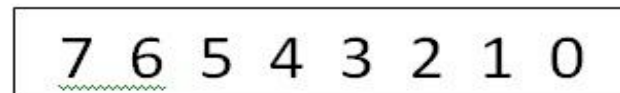
(1)



Register R

Fig. 4-1 a

(2)

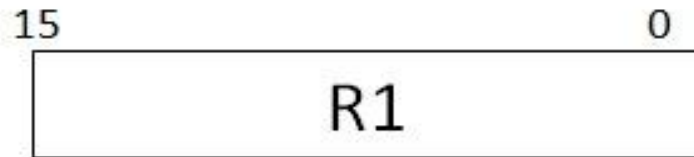


Individual Bits

Fig. 4-1 b

The numbering of bits in 16 bit register can be marked on the top of the box as shown in the fig **4-1 c**.

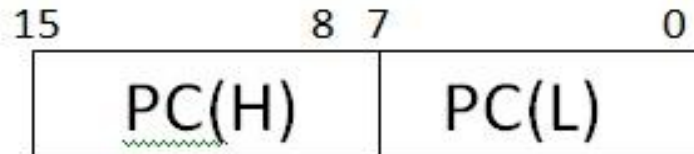
(3)



Numbering of Bits

Fig. 4-1 c

(4)



Two parts division of Register

Fig. 4-1 d

- A 16 bit register is divided into two parts in **4-1 d.** bits 0 through 7 are assigned the symbol L (for the low bytes) and bits 8 through 15 are assigned the symbol H(for high byte).
- The name of 16 bit register is PC.

REGISTER TRANSFER LANGUAGE

- A Language in which the symbolic notation is used to describe the micro operation transfer among registers is called a register transfer language.
- ‘**Register Transfer**’ means a logic circuit that performs micro operations and transfer the result of the operation to the same or another register.
- A Register transfer language is a system for expressing in symbolic form the micro-operation sequences among the registers of a digital module.

- Information transfer from one register to another is designated in symbol form by means of a replacement operator in the statement.

$$\mathbf{R2 \leftarrow R1}$$

- Denotes the transfer of the content of register R1 into register R2. By definition the content of the source register R1 does not change after the transfer.

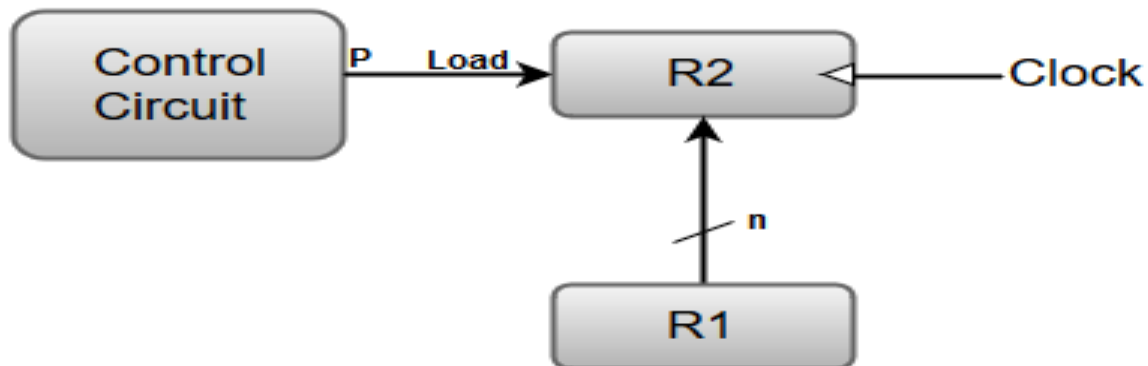
- Normally we want the transfer to occur only under a predetermined control condition. This can be shown by means of an If than statement

If ($P=1$) than ($R2 \leftarrow R1$)

- Where p is a control function which is a Boolean variable that is equal to 1 or 0 the control function in the statement as follows

$P: R2 \leftarrow R1$

Transfer from R1 to R2 when $P = 1$:



Symbols of Register Transfer :

Some symbols are used in Register Transfer Language to describe the expression.

Symbol	Description	Examples
Letters & Numerals	Denotes a Register	MAR, R1
Parentheses ()	Denotes a part of register	R1(0-7), R1(L)
Arrow $\leftarrow$	Denote to the transfer of information	$R1 \leftarrow R2$
Comma ,	Separate Micro-operation	$R1 \leftarrow R2, R2 \leftarrow R1$

BUS TRANSFER

- A BUS is the set of common lines (wires) in which each line is used for one bit of the register through which binary information is transferred at a time.
- A digital computer has many registers and path must be provided to transfer information from one register to another.

- The number of wires will be increased if separate lines are used between each register and all the other registers in the system.
- A more efficient scheme for transferring information between registers in a multiple register configuration in a common bus system.
- The bus structure contains of a set of common lines one for each bits of a register, through which binary information is transferred one at a time.

- There are two ways to construct common bus systems
- With the help of multiplexer
- With three state bus buffers

- The multiplexers select the source register whose binary information is then placed on the bus.
- Each register has four bits, numbered 0 through 3.
- The bus consists of four 4 x 1 multiplexers each having four data inputs, 0 through 3, and two selection inputs, S1 and S0.
- These two selection lines S1 and S0 are connected to the selection inputs of all four multiplexers.

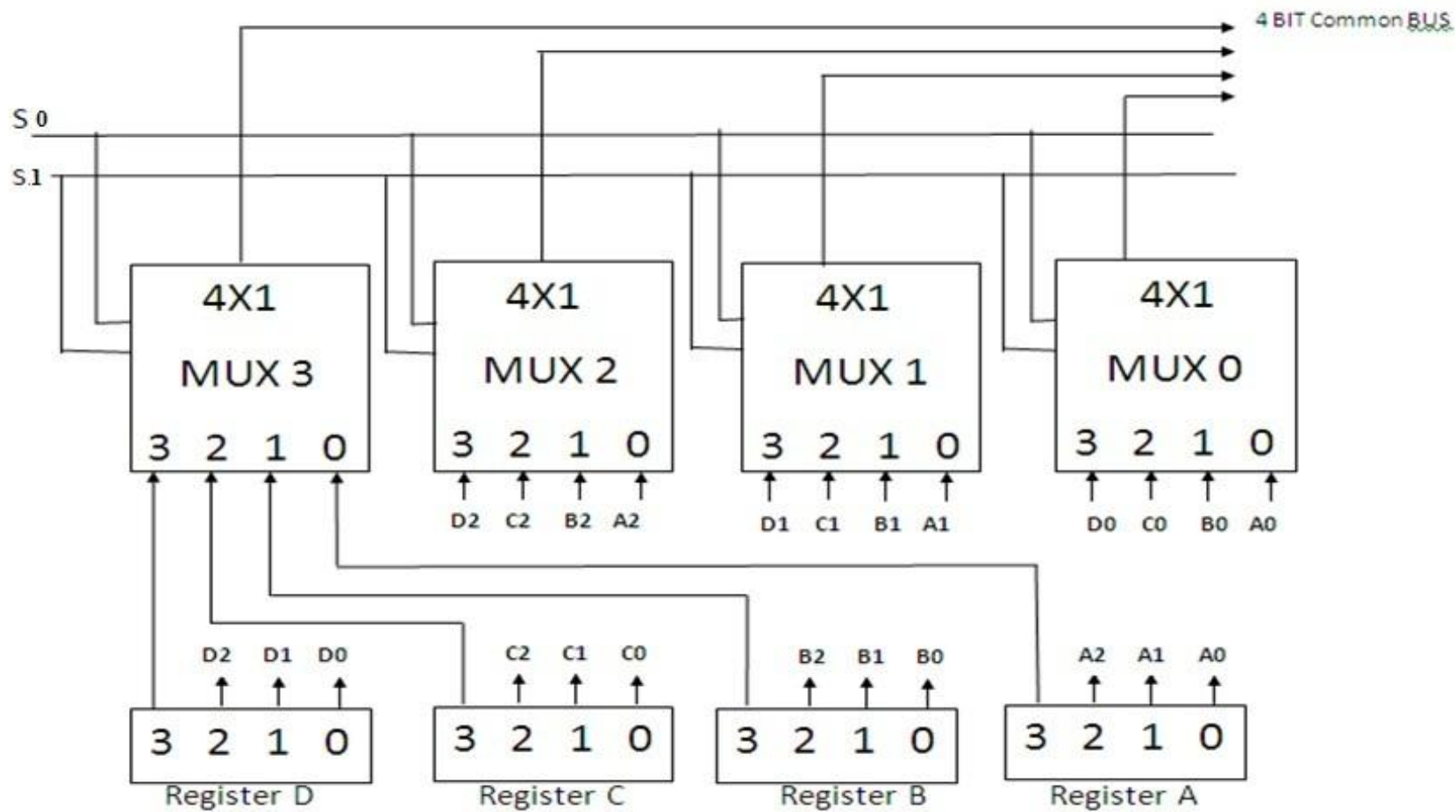


fig 4-3

- The selection lines choose the four bits of one register and transfer them into the four- line common bus.

S1	S0	Register Selected
0	0	A
0	1	B
1	0	C
1	1	D

- When $S_1S_0 = 00$, the 0 data inputs of all four multiplexers are selected and applied to the outputs that form the bus.
- This causes the bus lines to receive the content of register A since the outputs of this register are connected to the 0 data inputs of the multiplexers.
- Similarly, register B is selected if $S_1S_0 = 01$, and so on.

- A bus system will multiplex k registers of n bits each to produce an n -line common bus.
- The number of multiplexers needed to construct the bus is equal to n , the number of bits in each register.
- The size of each multiplexer must be $K \times 1$ since it multiplexes K data lines.

- Number of mux = Number of bits in the register.
- Number of inputs in Mux = Number of registers.

Question

- For a common bus for eight registers of 16 bits each requires how many mux?
- For a common bus system for eight registers what size of mux is required? And what number of selection lines are required?

Ans

- For instance, a common bus for eight registers of 16 bits each requires 16 multiplexers, one for each line in the bus. Each multiplexer must have eight data input lines and three selection lines to multiplex one significant bit in the eight registers.

Question

A digital computer has a common bus system of 32 registers of 64 bits each. The bus is constructed with multiplexer.

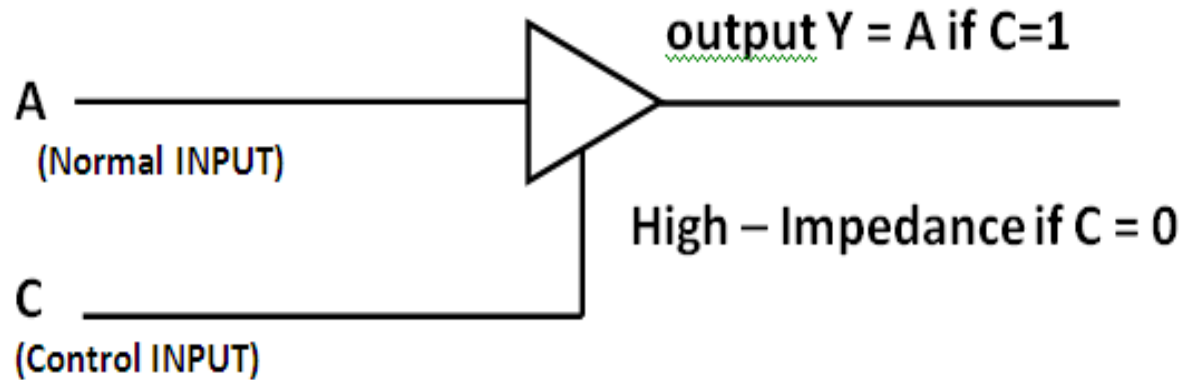
- a) How many selection inputs are there in each multiplexer?
- b) What size of multiplexers are needed?
- c) How many multiplexers are there in the bus?

Ans

- (a) 5 selection lines to select one of 32 registers.
- (b) 32×1 multiplexers.
- (c) 64 multiplexers, one for each bit of the registers.
-

Tri- state Buffer(3 state Buffer)

- A three state gate is a digital circuit that exhibits three states.
- Two of the states are signal equivalent to logic 1 and logic 0
- Third gate ,high-impedance gate.
- The most commonly used three state gate in case of bus system is a buffer gate.



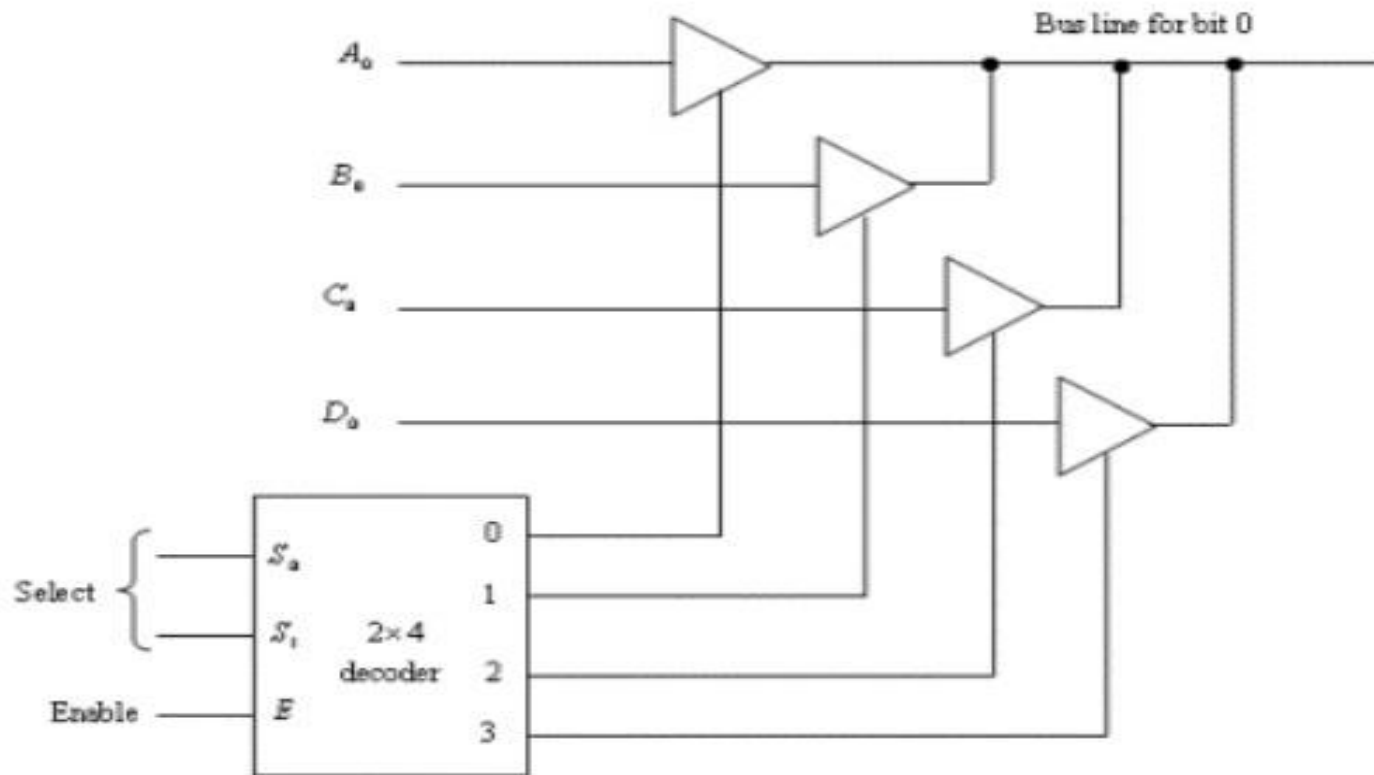
- The control input determines the output state. When the control input C is equal to 1, the output is enabled and the gate behaves like any conventional buffer, with the output equal to the normal input.
- When the control input C is 0, the output is disabled and the gate goes to a high- impedance state. Not show the result.

Common bus system using decoder and tri state buffer

- A bus system can be constructed with three state gates instead of multiplexer.
- The outputs of four buffers are connected together to form a single bus line.
- The control inputs to the buffers determine which of the four normal inputs will communicate with the bus line.
- No more than one buffer may be in the active state at any given time.

- The connected buffers must be controlled so that only one three-state buffer has access to the bus line while all other buffers are maintained in a high impedance state.

BUS LINE WITH THREE STATE BUFFER



- One way to ensure that no more than one control input is active at any given time is to use a decoder
- When the enable input of the decoder is 0, all of its four outputs are 0, and the bus line is in a high-impedance state because all four buffers are disabled.
- When the enable input is active, one of the three-state buffers will be active, depending on the binary value in the select inputs of the decoder.
- To construct a common bus for four registers of n bits each using three-state buffers, we need n circuits with four buffers in each as shown in figure: Bus line with three state-buffers,

MEMORY TRANSFER

- **Read Operation:** The transfer of information from a memory word to the outside environment is called a read operation.
- **Write Operation:** The transfer of new information to be stored into the memory is called a write operation.
- A memory word will be symbolized by the letter M.
- This will be done by enclosing the address in square brackets following the letter M.

Memory Read: Consider a memory unit that receives the address from a register, called a register denoted by **AR**. The data are transferred to another register called data register denoted by **DR**. The read operation can be stated as follows:-

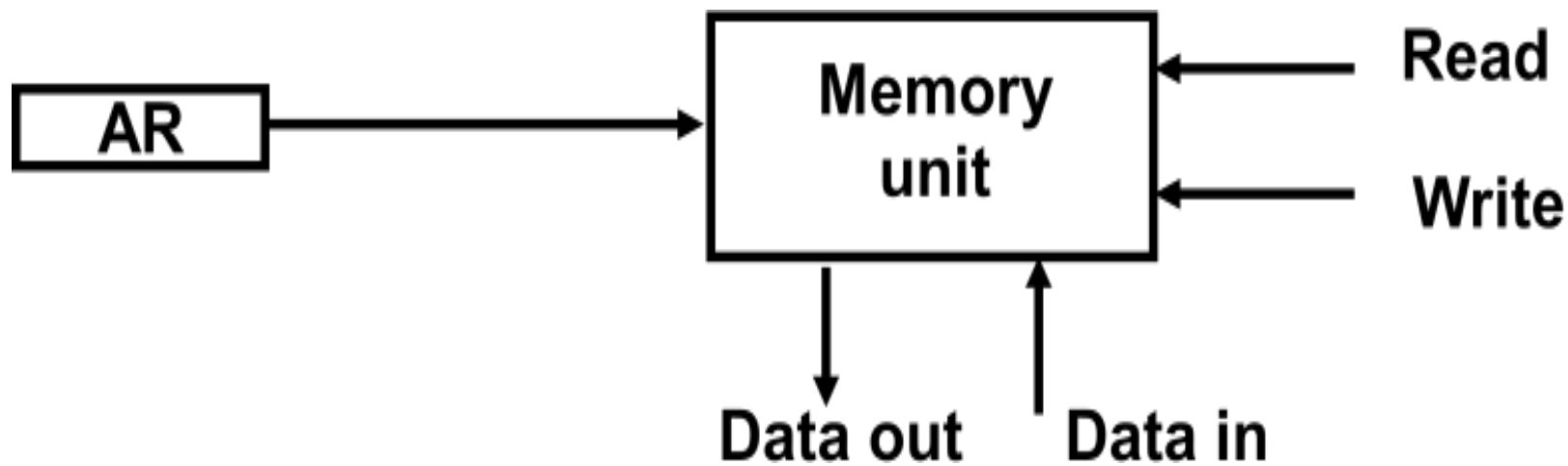
$$\text{Read: DR} \leftarrow \text{M[AR]}$$

This causes a transfer of information into DR from word M selected by the address in AR.

- **Memory Write:** The write operation transfers the content of a data register to a memory word M selected by the address. Consider that input data are in register $R1$ and address is in AR . The write operation can be stated symbolically as follows:

Write: $M[AR] \leftarrow R1$

- This causes a transfer of information from $R1$ into the memory word M selected by address in AR .



Micro operations

- Digital systems are modular in nature, with modules containing registers, decoders, arithmetic elements, control logic, etc.
- These digital components are defined by the registers that they contain and the operations performed on their data. These operations are called micro operations.
- Micro operations are elementary operations performed on the information stored in one or more registers.

Micro operations are classified into four categories:

- – Register transfer micro operations (data moves from register to register)
- – Arithmetic micro operations (perform arithmetic on data in registers)
- – Logic micro operations (perform bit manipulation on data in registers)
- – Shift micro operations (perform shift on data in registers)

Arithmetic Microoperations

- Unlike register transfer microoperations, arithmetic microoperations change the information content.
- The basic arithmetic microoperations are:
 - addition
 - subtraction
 - increment
 - decrement
 - shift

Arithmetic Microoperations (continued)

- The RTL statement:
- $R3 \leftarrow R1 + R2$
- indicates an add microoperation. We can similarly specify the other arithmetic microoperations.
- Multiplication and division are not considered microoperations.
 - Multiplication is implemented by a sequence of adds and shifts.
 - – Division is implemented by a sequence of subtracts and shifts.

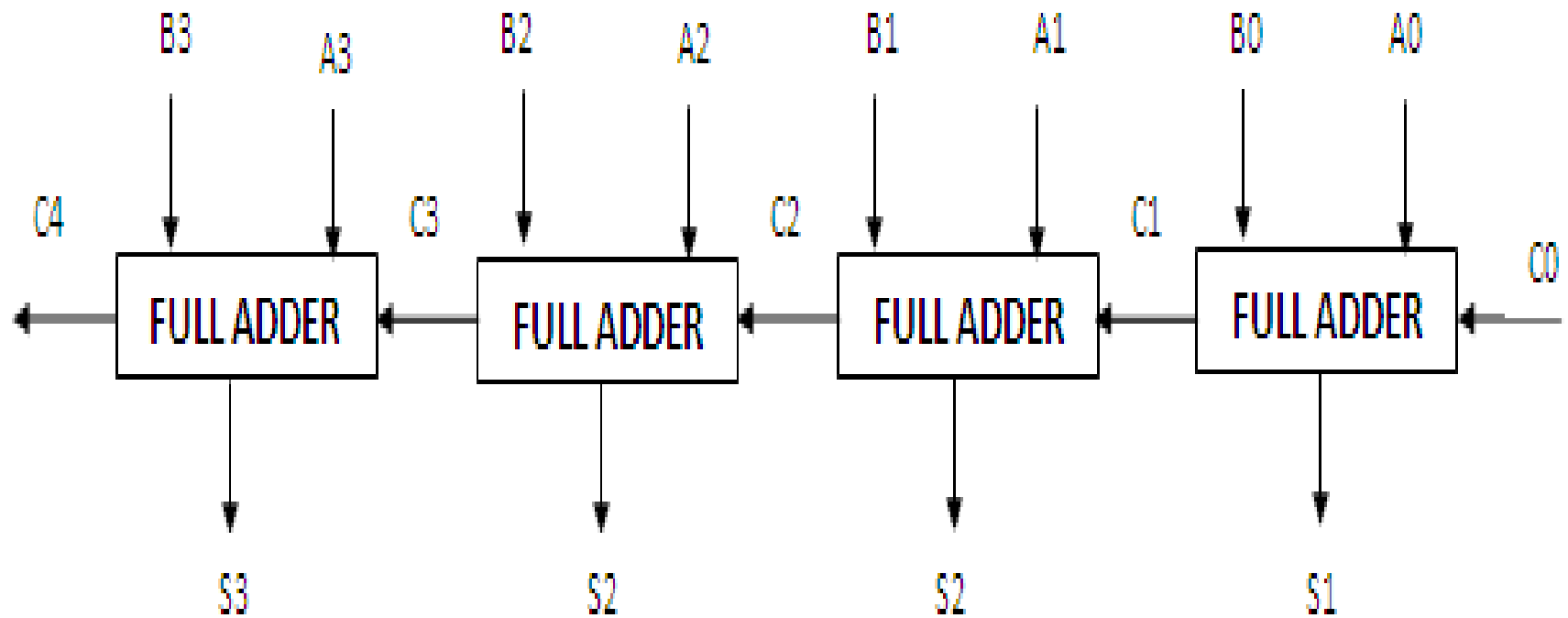
Arithmetic Microoperations

<u>Symbolic Designation</u>	<u>Description</u>
$R3 \leftarrow R1 + R2$	Contents of R1 plus R2 transferred to R3
$R3 \leftarrow R1 - R2$	Contents of R1 minus R2 transferred to R3
$R2 \leftarrow R2'$	Complement contents of R2 (1's comp.)
$R2 \leftarrow R2' + 1$	2's complement contents of R2 (negate)
$R3 \leftarrow R1 + R2' + 1$	R1 plus 2's comp. of R2
$R1 \leftarrow R1 + 1$	Increment content of R1 by 1
$R1 \leftarrow R1 - 1$	Decrement content of R1 by 1

Binary Adder

- We implement a binary adder with registers to hold the data and a digital circuit to perform the addition (called a **binary adder**).
- The binary adders is constructed using full adders connected in cascade so that the carry produced by one full adder becomes an input for the next.
- Adding two n-bit numbers requires n full adders.
- The n data bits for **A** and **B** might come from **R1** and **R2** respectively

4-Bit Binary Adder



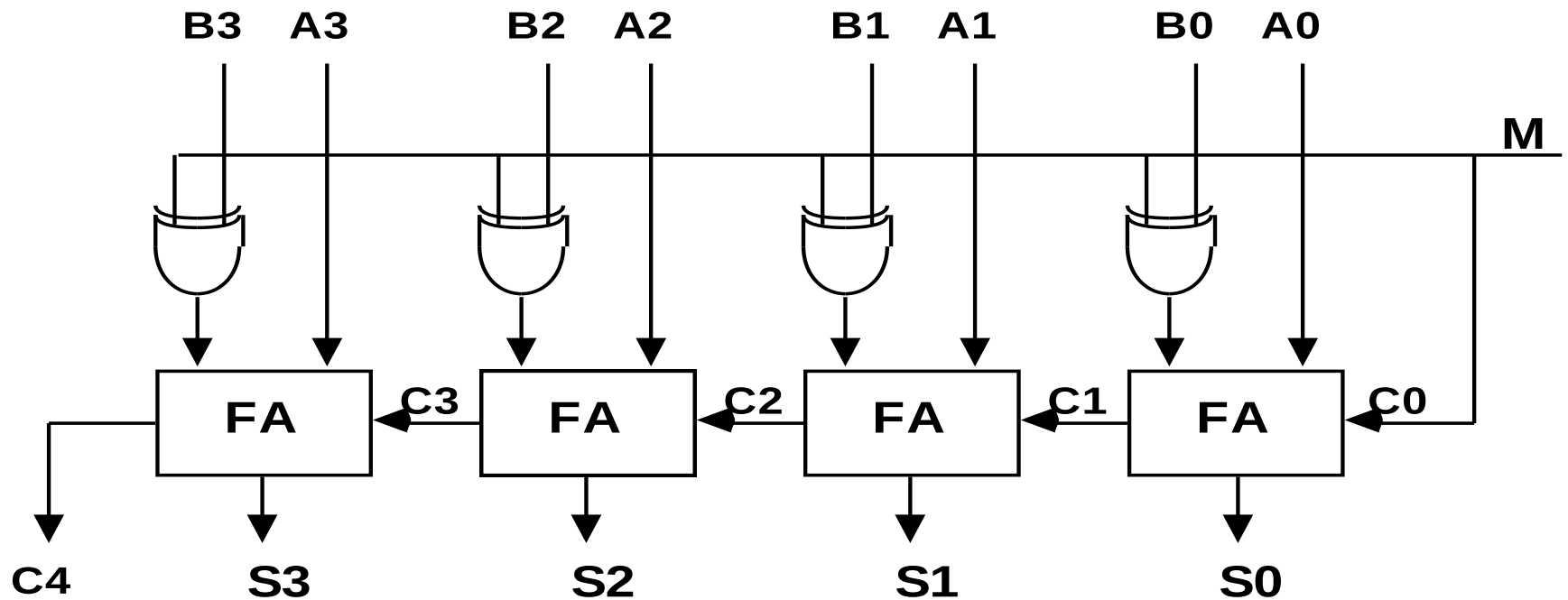
Subtract micro operation :

- This micro operation subtracts the content of one register from content of another register and result is transferred into third register.
- In computer systems, the subtraction is performed through complement and addition of the contents.
- if two's complement of a number is added with the another number then its subtraction operation is performed in the computer.
- For example: $R3 \leftarrow R1 + R2' + 1$
- Adding to the content of the **R1** with **2's** complement of **R2** is equivalent to **R1 – R2**, in the above statement **R2'** is the 1's complement and **1** is added to find 2's complement of R2.

Adder-Subtractor

- Subtracting $A - B$ is most easily done by adding B' to A and then adding 1.
- This makes it convenient to combine both addition and subtraction into one circuit, called an adder-subtractor.
- M is the mode indicator
 - $M = 0$ indicates addition (B is left alone and C_0 is 0)
 - $M = 1$ indicates subtraction (B is complement and C_0 is 1).

each **XOR** gate receives input M and one of the inputs B . When $M=0$ we have $B \text{ XOR } 0 = B$. when $M=1$, we have $B \text{ XOR } 1 = B'$ and $C0=1$.

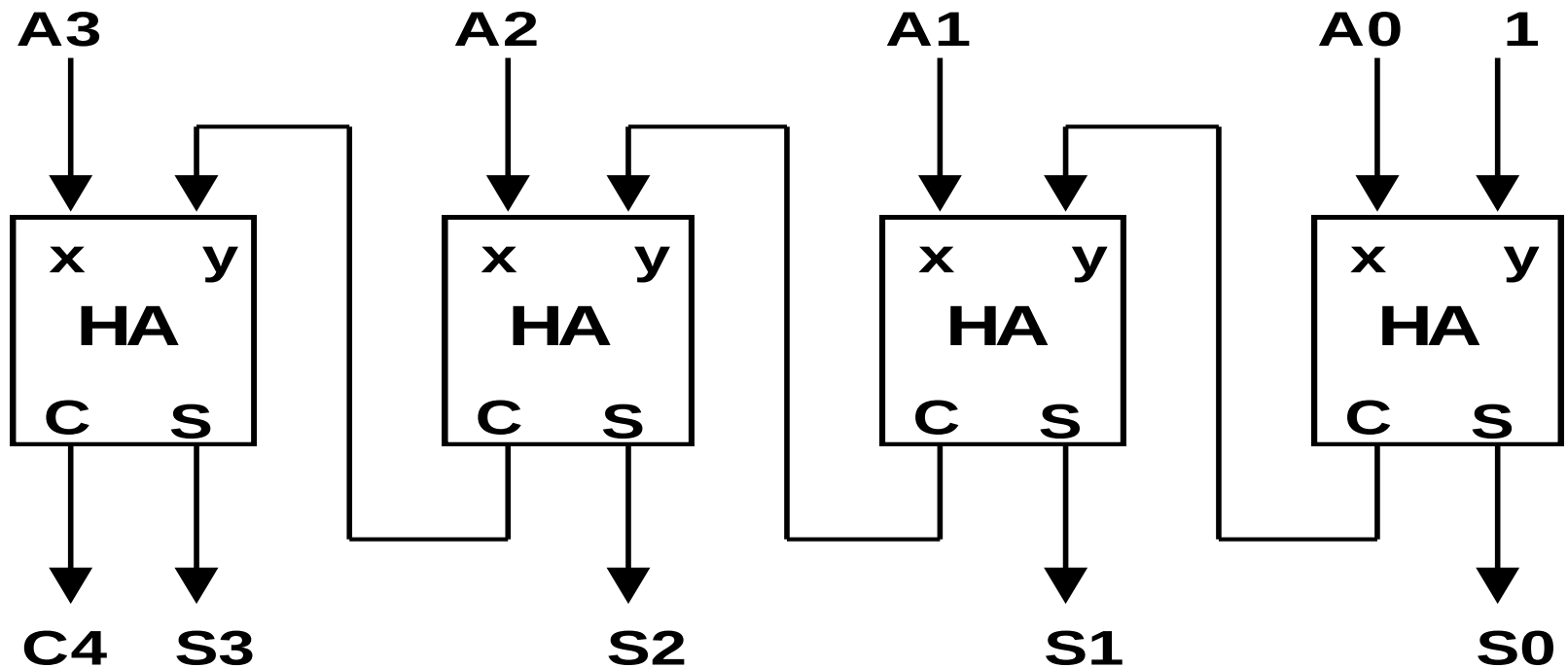


Increment microoperation :

This microoperation increments the one value of register.

- For example:- $R1 \leftarrow R1 + 1$
- Binary incrementer
- The binary incrementer adds 1 to the contents of a register, e.g., a register storing 0101 would have 0110 in it after being incremented.
- There are times when we want incrementing done independent of a register. We can accomplish this with a series of cascading half-adders.

The circuit receives the four bits A0 to A3 adds one to it and generates the incremented output S0 through S3.



Decrement Microoperation: This microoperation decrement one values from value of register.

For example: $R1 \leftarrow R1 - 1$

ARITHMETIC CIRCUIT

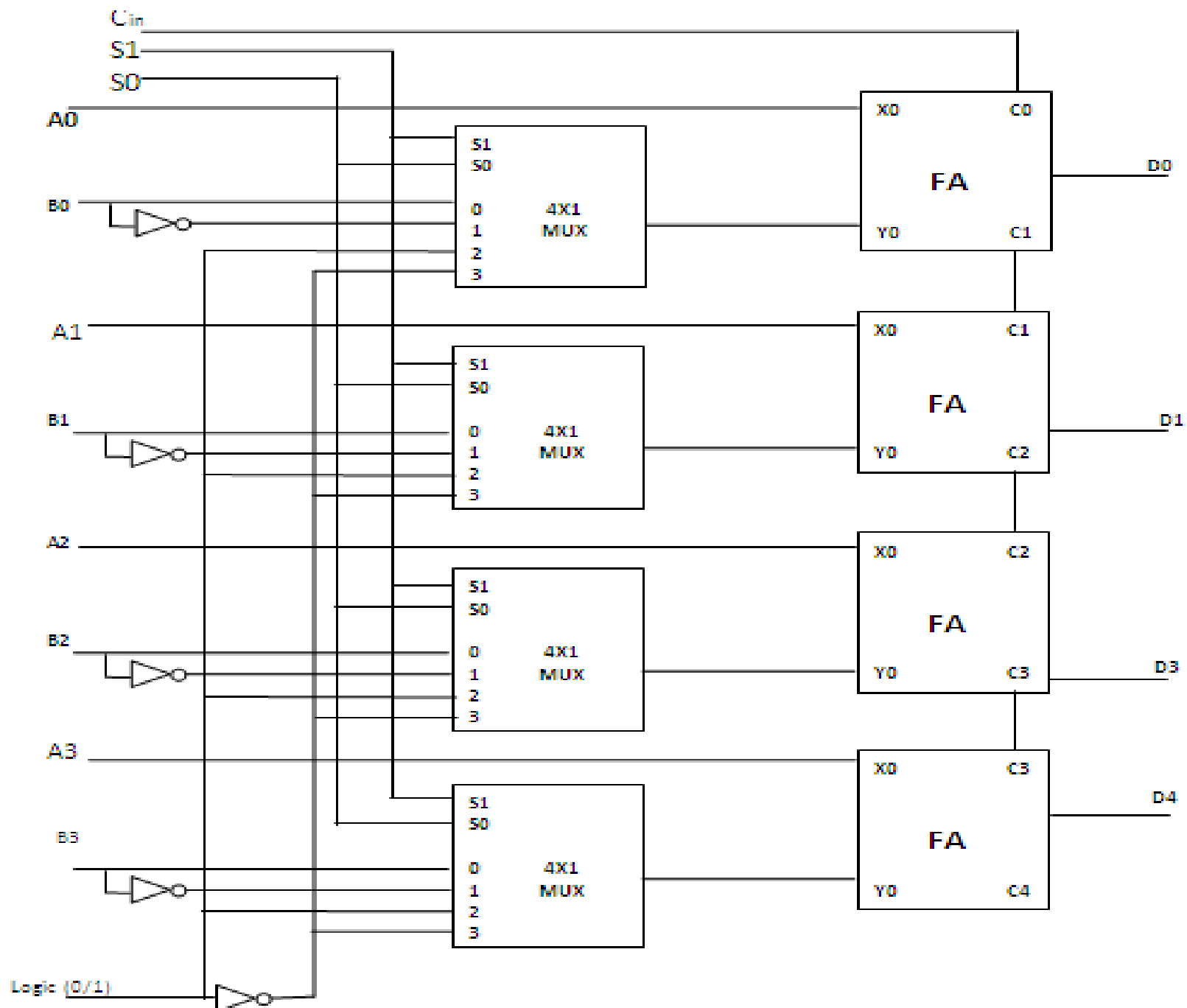
- Arithmetic circuit can be implemented in one composite arithmetic circuit. the basic component of an arithmetic circuit is parallel adder.
- We can implement 7 arithmetic micro-operations (add, add with carry, subtract, subtract with borrow, increment, decrement and transfer) with one circuit
- Diagram of 4 bit parallel circuit is shown. This circuit is used for all arithmetic Micro operation in which 4bit binary adder and 4 multiplexer are used.

- In each full adder , two inputs X & y are passed. The four A input is passed to the X input of full adder and four B input is passed to the Y input of each full adder through multiplexer A and B are 4bit numbers where A has $A_3A_2A_1A_0$ and B has $B_3B_2B_1B_0$.
- The four inputs of B has its normal and complement value which is input into 0, 1 position input of each multiplexer .the input 0 and 1 is passed to the 2,3 position of each multiplexer.
- A carry C_{in} is passed to each full adder of 4 bit binary adder.

The four multiplexer uses two common selection lines s_0, s_1 to select the number of input value. In this arithmetic circuit the output of binary adder is calculated from the expression given below:-

Table : Arithmetic Circuit Function Table

SELECT			Input Y	Output $D = A + Y + C_{in}$	Microoperation
S1	S0	C_{in}			
0	0	0	B	$D = A + B$	Add
0	0	1	B	$D = A + B + 1$	Add with Carry
0	1	0	B'	$D = A + B'$	Subtract with borrow
0	1	1	B'	$D = A + B' + 1$	Subtract
1	0	0	0	$D = A$	Transfer A
1	0	1	0	$D = A + 1$	Increment A
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A



- When $S_1S_0 = 00$, the MUX provides B. The result is Add (for $C_{in} = 0$) or Add With Carry (for $C_{in} = 1$).
- When $S_1S_0 = 01$, the MUX provides B'. The result is Subtract with Borrow (for $C_{in} = 0$) or Subtract (for $C_{in} = 1$).
- When $S_1S_0 = 10$, the MUX provides 0. The result is Transfer (for $C_{in} = 0$) or Increment (for $C_{in} = 1$).
- When $S_1S_0 = 11$, the MUX provides 1. The result is Decrement (for $C_{in} = 0$) or Transfer (for $C_{in} = 1$).

Logic Microoperations

- Logic microoperations are binary operations performed on corresponding bits of two bit strings.
- Example: P: $R1 \leftarrow R1 \oplus R2$
1010 Content of R1
1100 Content of R2
0110 Content of R1 after P = 1
- Special Symbols used for logic operations:
 $\wedge$ - AND $\vee$ - OR $\oplus$ - XOR

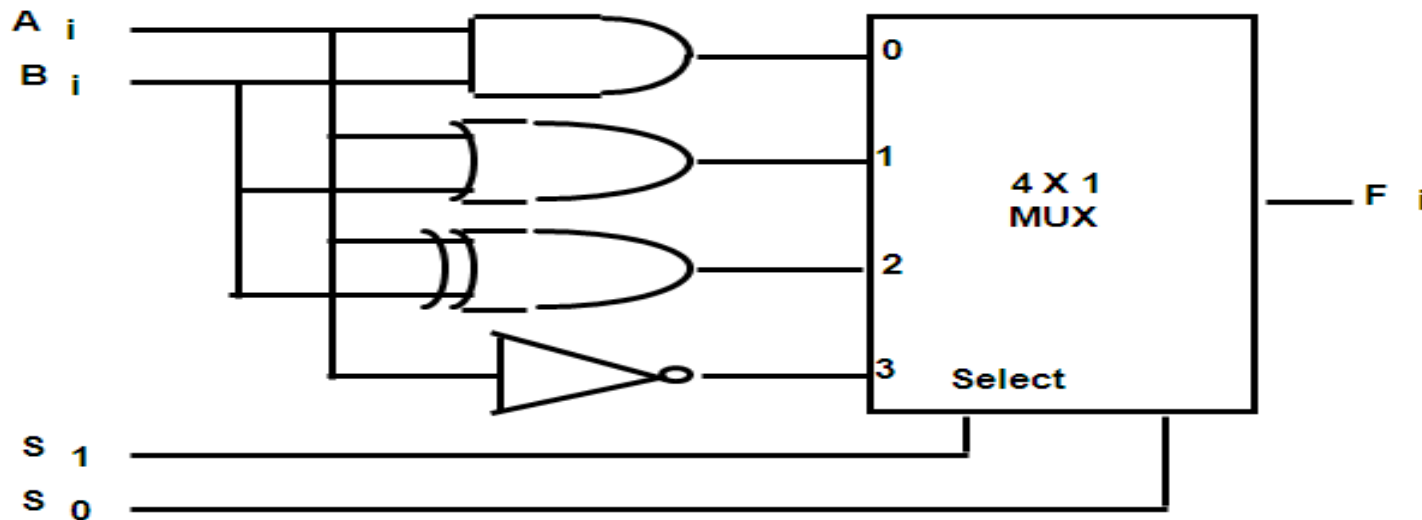
- Logic micro operations are bit-wise operations, i.e., they work on the individual bits of data.
- useful for bit manipulations on binary data.
- useful for making logical decisions based on the bit value

- 16 different logic operations with 2 binary vars.
- Truth tables for 16 functions of 2 variables and the corresponding 16 logic micro-operations.

x	0 0 1 1	Boolean Function	Micro- Operations	Name
y	0 1 0 1			
	0 0 0 0	$F0 = 0$	$F \leftarrow 0$	Clear
	0 0 0 1	$F1 = xy$	$F \leftarrow A \wedge B$	AND
	0 0 1 0	$F2 = xy'$	$F \leftarrow A \wedge B'$	
	0 0 1 1	$F3 = x$	$F \leftarrow A$	Transfer A
	0 1 0 0	$F4 = x'y$	$F \leftarrow A' \wedge B$	
	0 1 0 1	$F5 = y$	$F \leftarrow B$	Transfer B
	0 1 1 0	$F6 = x \oplus y$	$F \leftarrow A \oplus B$	Exclusive-OR
	0 1 1 1	$F7 = x + y$	$F \leftarrow A \vee B$	OR
	1 0 0 0	$F8 = (x + y)'$	$F \leftarrow (A \vee B)'$	NOR
	1 0 0 1	$F9 = (x \oplus y)'$	$F \leftarrow (A \oplus B)'$	Exclusive-NOR
	1 0 1 0	$F10 = y'$	$F \leftarrow B'$	Complement B
	1 0 1 1	$F11 = x + y'$	$F \leftarrow A \vee B$	
	1 1 0 0	$F12 = x'$	$F \leftarrow A'$	Complement A
	1 1 0 1	$F13 = x' + y$	$F \leftarrow A' \vee B$	
	1 1 1 0	$F14 = (xy)'$	$F \leftarrow (A \wedge B)'$	NAND
	1 1 1 1	$F15 = 1$	$F \leftarrow \text{all 1's}$	Set to all 1's

Draw a combinational circuit that perform four logical micro operations.

HARDWARE IMPLEMENTATION OF LOGIC MICROOPERATIONS



Function table

S_1	S_0	Output	μ -operation
0	0	$F = A \wedge B$	AND
0	1	$F = A \vee B$	OR
1	0	$F = A \oplus B$	XOR
1	1	$F = A'$	Complement

Applications of logic microoperations

- Logic microoperations can be used to manipulate individual bits or a portions of a word in a register
- Consider the data in a register A. In another register, B, is bit data that will be used to modify the contents of A

- Selective-set

$$A \leftarrow A + B$$

- Selective-complement

$$A \leftarrow A \oplus B$$

- Selective-clear

$$A \leftarrow A \cdot B'$$

- Mask (Delete)

$$A \leftarrow A \cdot B$$

- Clear

$$A \leftarrow A \oplus B$$

- Insert

$$A \leftarrow (A \cdot B) + C$$

- Compare

$$A \leftarrow A \oplus B$$

SELECTIVE SET

- In a selective set operation, the bit pattern in B is used to *set* certain bits in A

1 1 0 0 A_t

1 0 1 0 B

1 1 1 0 A_{t+1} ($A \leftarrow A + B$)

- If a bit in B is set to 1, that same position in A gets set to 1, otherwise that bit in A keeps its previous value

SELECTIVE COMPLEMENT

- In a selective complement operation, the bit pattern in B is used to *complement* certain bits in A

1 1 0 0 A_t

1 0 1 0 B

0 1 1 0 $A_{t+1} \quad (A \leftarrow A \oplus B)$

- If a bit in B is set to 1, that same position in A gets complemented from its original value, otherwise it is unchanged

SELECTIVE CLEAR

- In a selective clear operation, the bit pattern in B is used to *clear* certain bits in A

$$\begin{array}{ll} 1\ 1\ 0\ 0 & A_t \\ 1\ 0\ 1\ 0 & B \\ 0\ 1\ 0\ 0 & A_{t+1} \quad (A \leftarrow A \cdot B') \end{array}$$

- If a bit in B is set to 1, that same position in A gets set to 0, otherwise it is unchanged

MASK OPERATION

- In a mask operation, the bit pattern in B is used to *clear* certain bits in A

$$\begin{array}{ll} 1\ 1\ 0\ 0 & A_t \\ 1\ 0\ 1\ 0 & B \\ 1\ 0\ 0\ 0 & A_{t+1} \quad (A \leftarrow A \cdot B) \end{array}$$

- If a bit in B is set to 0, that same position in A gets set to 0, otherwise it is unchanged

CLEAR OPERATION

- In a clear operation, if the bits in the same position in A and B are the same, they are cleared in A, otherwise they are set in A

1 0 1 0 A_t

1 0 1 0 B

0 0 0 0 $A_{t+1} \quad (A \leftarrow A \oplus B)$

INSERT OPERATION

- An Insert operation is used to introduce a specific bit pattern into A register, leaving the other bit positions unchanged
- This is done as
 - A mask operation to clear the desired bit positions, followed by
 - An OR operation to introduce the new bits into the desired positions

– Example

- Suppose you wanted to introduce 1010 into the low order four bits of A: 1101 1000 1011 0001 A (Original)
1101 1000 1011 1010 A (Desired)

- 1101 1000 1011 0001 A (Original)
1111 1111 1111 0000 Mask
1101 1000 1011 0000 A (Intermediate)
0000 0000 0000 1010 Added bits
1101 1000 1011 1010 A (Desired)

Shift micro operations

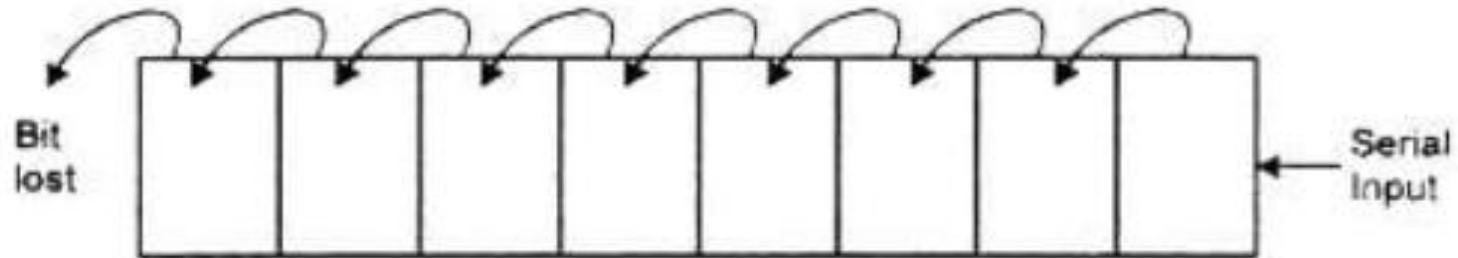
- Shift micro operations are used for serial transfer of data.
- This operations shift contents of register in left or right direction.
- At the same time when the bits are shifted the last or flip-flop receives binary information from serial inputs.
- There are three types of shift operations:
 - Logical shift micro operation.
 - Circular shift micro operation
 - Arithmetic Shift micro operation

Symbolic Designation	Description
$R \leftarrow \text{shl } R$	Shift-left register R
$R \leftarrow \text{shr } R$	Shift-right register R
$R \leftarrow \text{cil } R$	Circular shift-left register R
$R \leftarrow \text{cir } R$	Circular shift-right register R
$R \leftarrow \text{ashl } R$	Arithmetic Shift-left register R
$R \leftarrow \text{ashr } R$	Arithmetic Shift-right register R

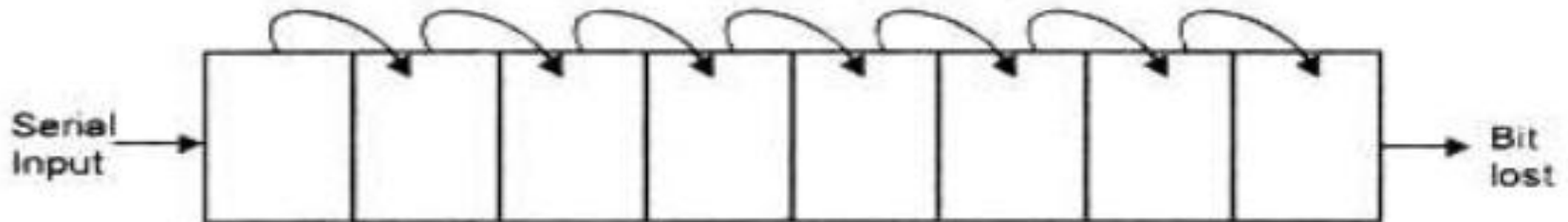
Logical Shift

- The standard logical shifts shift all bits one bit position to the left or right and place a 0 in the leftmost or rightmost position, respectively.
- It can be defined in RTL by:
- $R \leftarrow \text{shl } R$ shift-left register R
- $R \leftarrow \text{shr } R$ shift-right register R

example



The bits are shifted to words **left** by one bit position

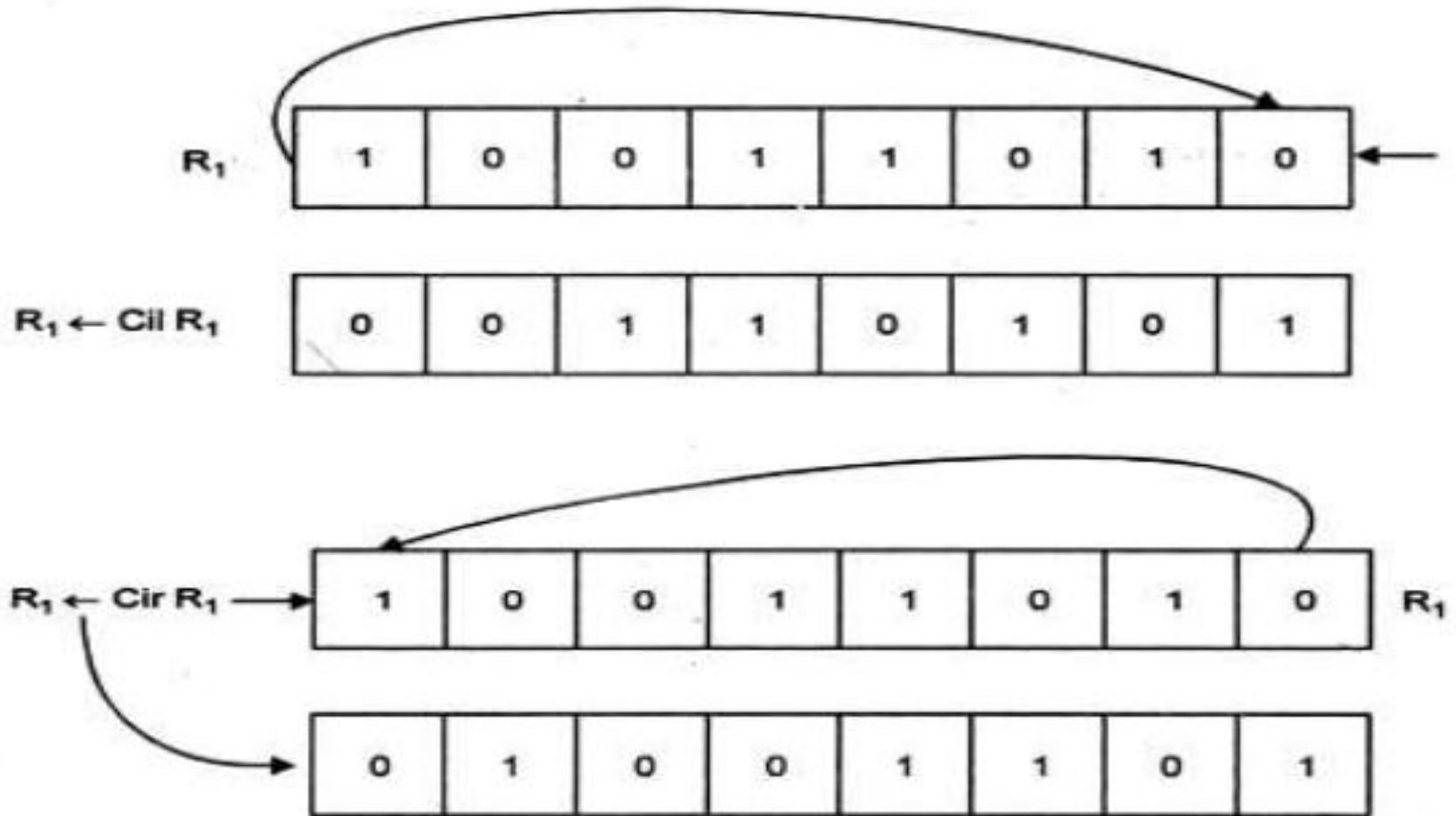


The bits are shifted to words **right** by one bit position

Circular shift

- The circular shift is also known as Rotate micro operation. It circulates the bit around both ends without any loss of information.
- It operate the same as a logical shifts except that instead of shifting a 0 into the end bit, the leftmost bit is shifted into the rightmost bit position on a CIL, and the rightmost bit is shifted into the leftmost bit position on a CIR.
- It can be defined in RTL by:
 $R \leftarrow cil\ l$ circular shift-left register R
 $R \leftarrow cir\ r$ circular shift-right register R

Circular shift

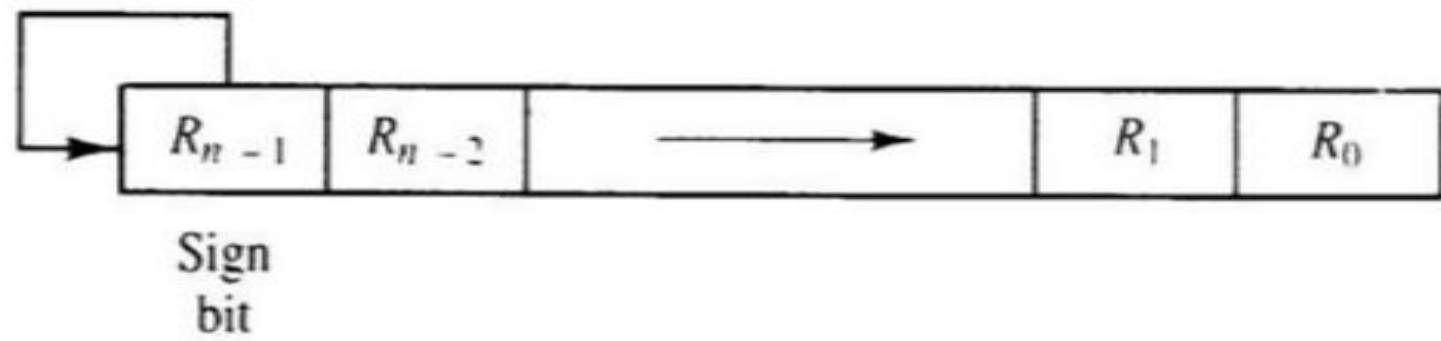


Arithmetic shift

- The arithmetic shift shifts a signed binary number to the left or right
- To the left is multiplying by 2, to the right is dividing by 2
- Arithmetic shifts must leave the sign bit unchanged
- Because the sign of the number remain same when it multiplied or divided by 2.
- The left bit hold the sign bit and others hold number
- The sign bit 0 for positive and 1 for negative
- It can be defined in RTL by:

$R \leftarrow \text{ashl } 1$ arithmetic shift-left register R

$R \leftarrow \text{ashr } r$ arithmetic shift-right register R



Instruction Code

- **Program**

- A program is a set of instructions that specify the operation operands and the sequence by which processing has to occur.

- **Computer Instruction**

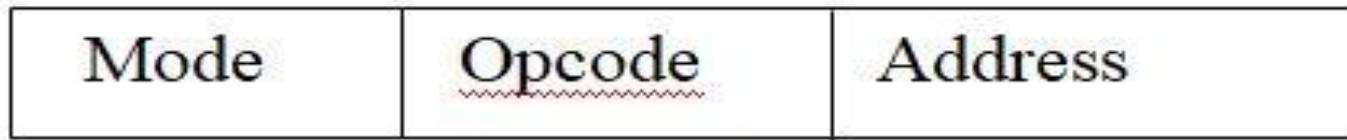
- A computer instruction is a binary code that specifies a sequence of microoperations for the computer.
- The computer reads the each instruction from memory and places it in a control register.
- The control register interpret the binary code of instruction and proceeds to execute it by issuing a sequence of microoperation.

Instruction Code

- An instruction code is a group of bits that instruct the computer to perform a specific operation.

Example : ADD457

- Instruction is usually divided into three parts, each having interpretation.



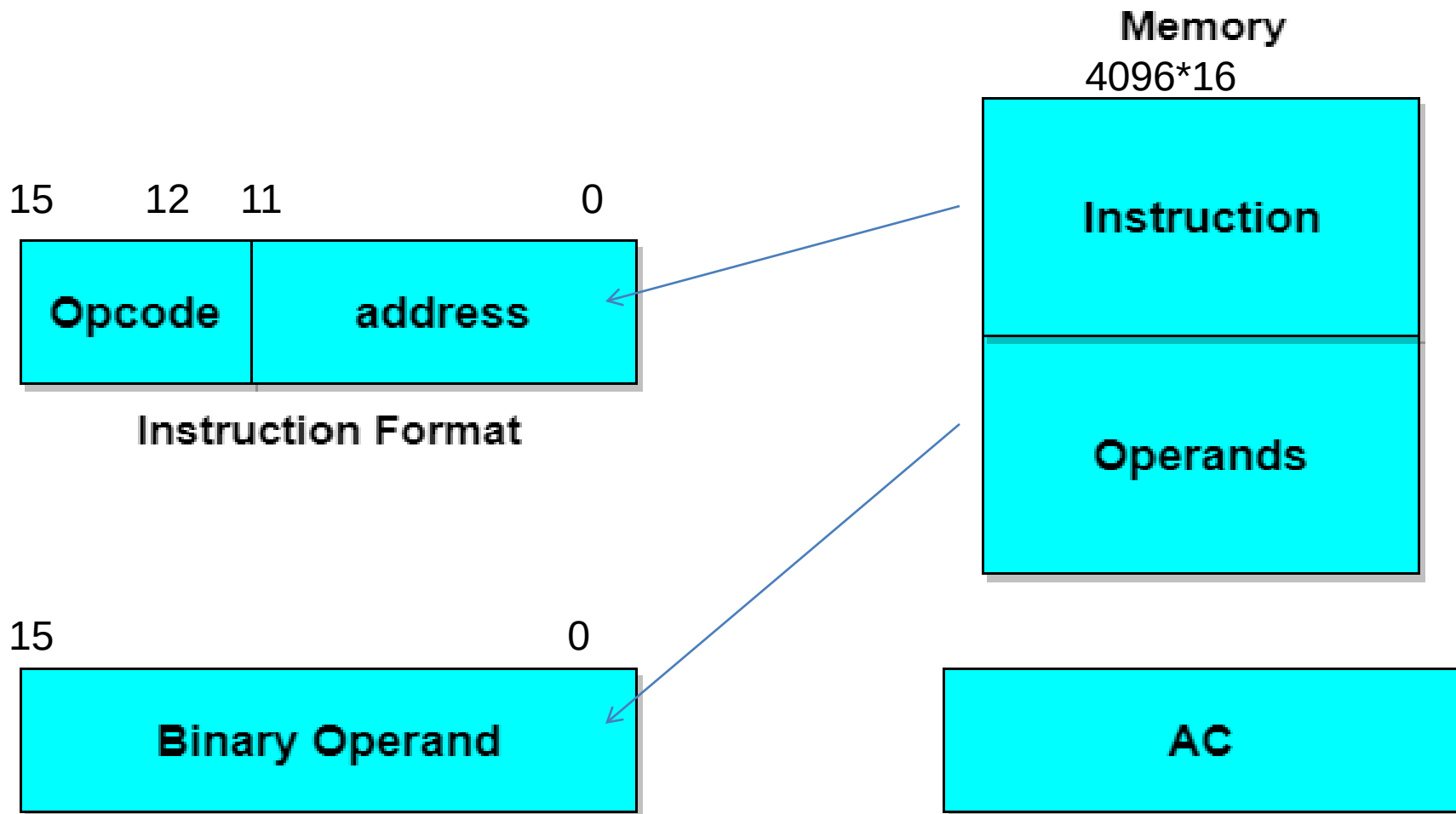
Instruction Format

- **Operation code(Opcode)**

- Operation code of an instruction is a group of bits that define such operation as add, subtract ,multiply , shift and complement.
- The number of bits required for the operation code of an instruction depends on the total number of operations available in computer
- the operation code must contains n bit for given 2^n (or less) different operation.

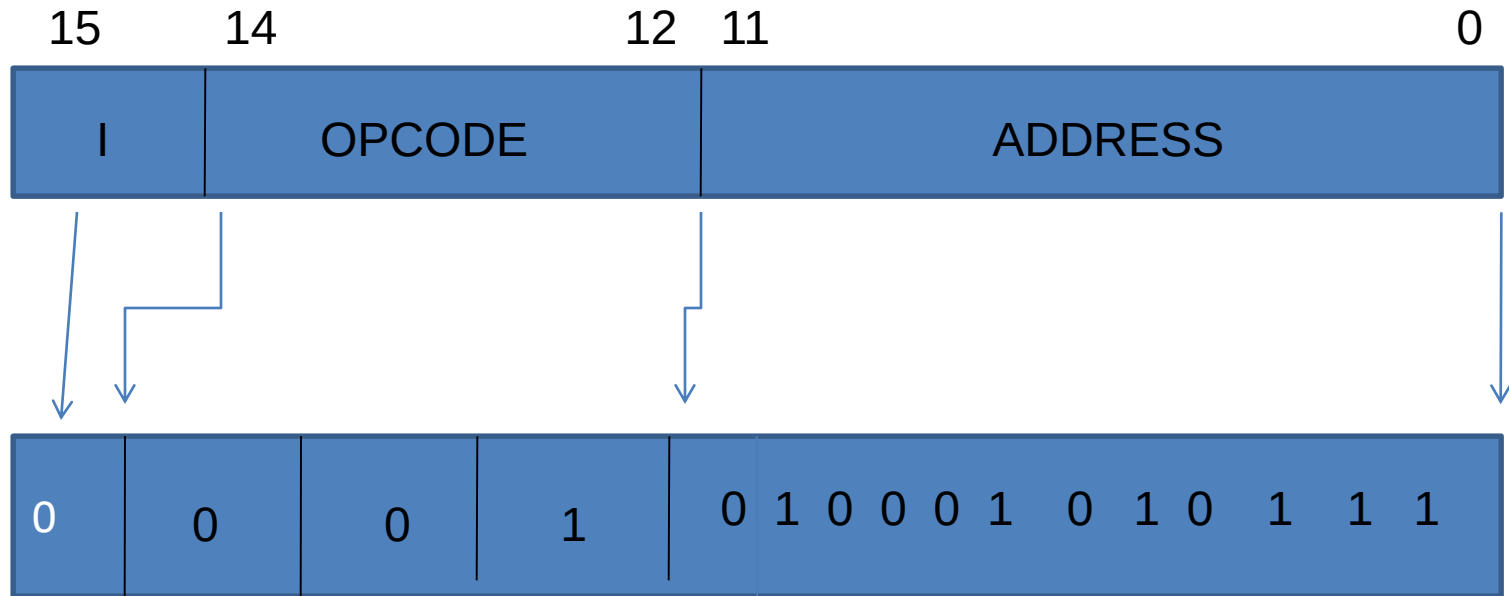
- **For example :**
- Consider a computer with 64 distinct operations, one of them being an ADD operation, the Opcode consist of 6 bits with a bit configuration are grant to ADD operation is 110010.
- “An operation code is some time called a Macro-operation because it specifies a set of micro operations.”

- **Address**
- This part of the instruction specifies the address of the operand.
- **Example:**
- Instruction are stored in one section of memory and data in another. for a memory unit with 4096 words. there are 12 bits are required to specify address($2^{12}=4096$).if we store each instruction code in one 16 bit memory word then 4 bits are required for the operation code.



Instruction format of basic computer

- Instruction format



Add Instruction –ADD457

Address is of two types:

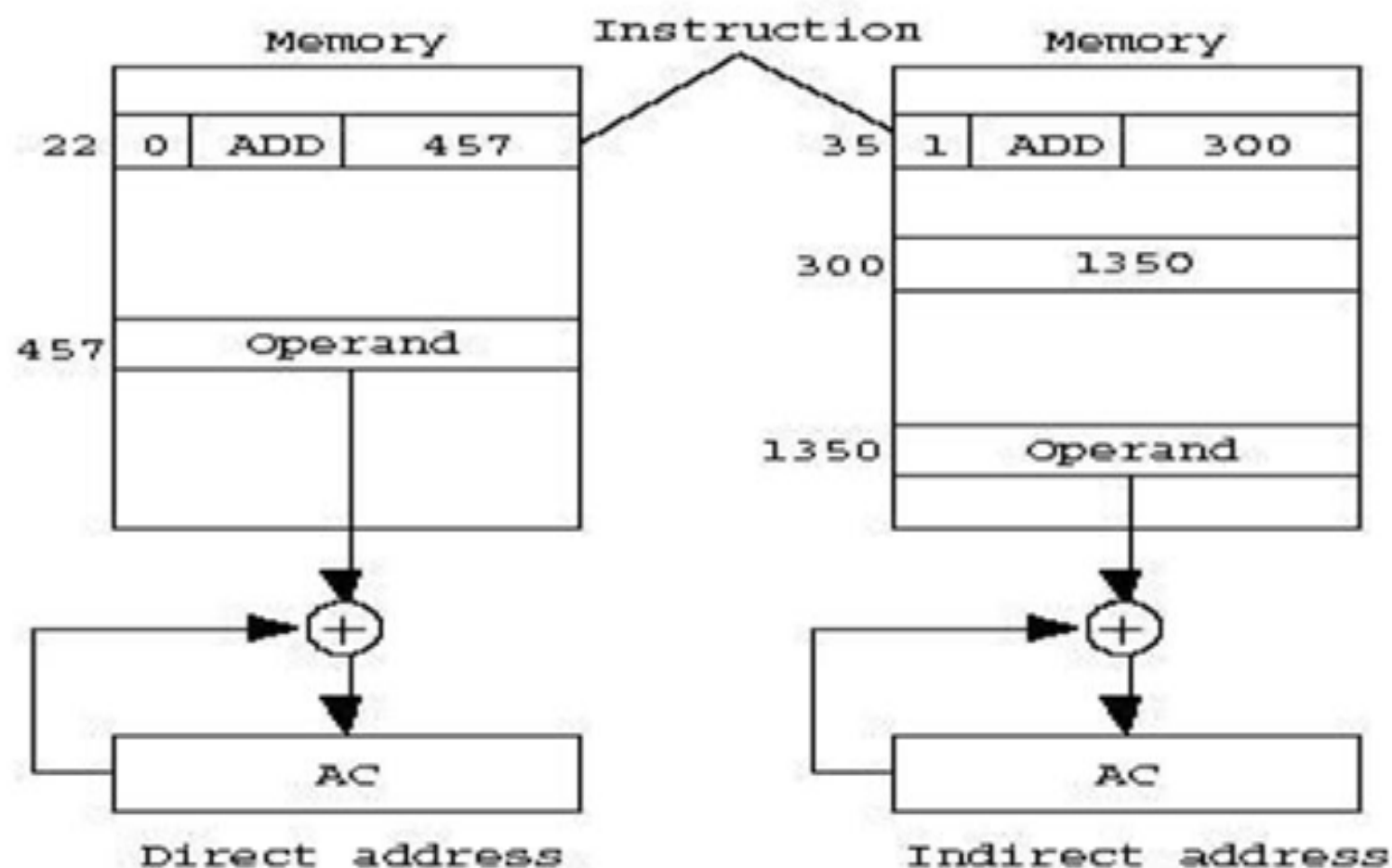
Direct address

- When the second part of instruction code specifies the address of an operand, instruction is said to have direct address.
- it can be identified with the value of I. if $I=0$ then address is direct address.

- **Indirect address**
- When the second part of the instruction code specifies the address of memory word in which address of operand are specified.
- it can be identified with the value of I. if $I=1$ then address is indirect address.
- **Effective address:-**
- It is the address of operand where operand has been stored. In direct address ,the effective address is 457 and in indirect address, effective address is 1350.



Instruction format



Computer Registers

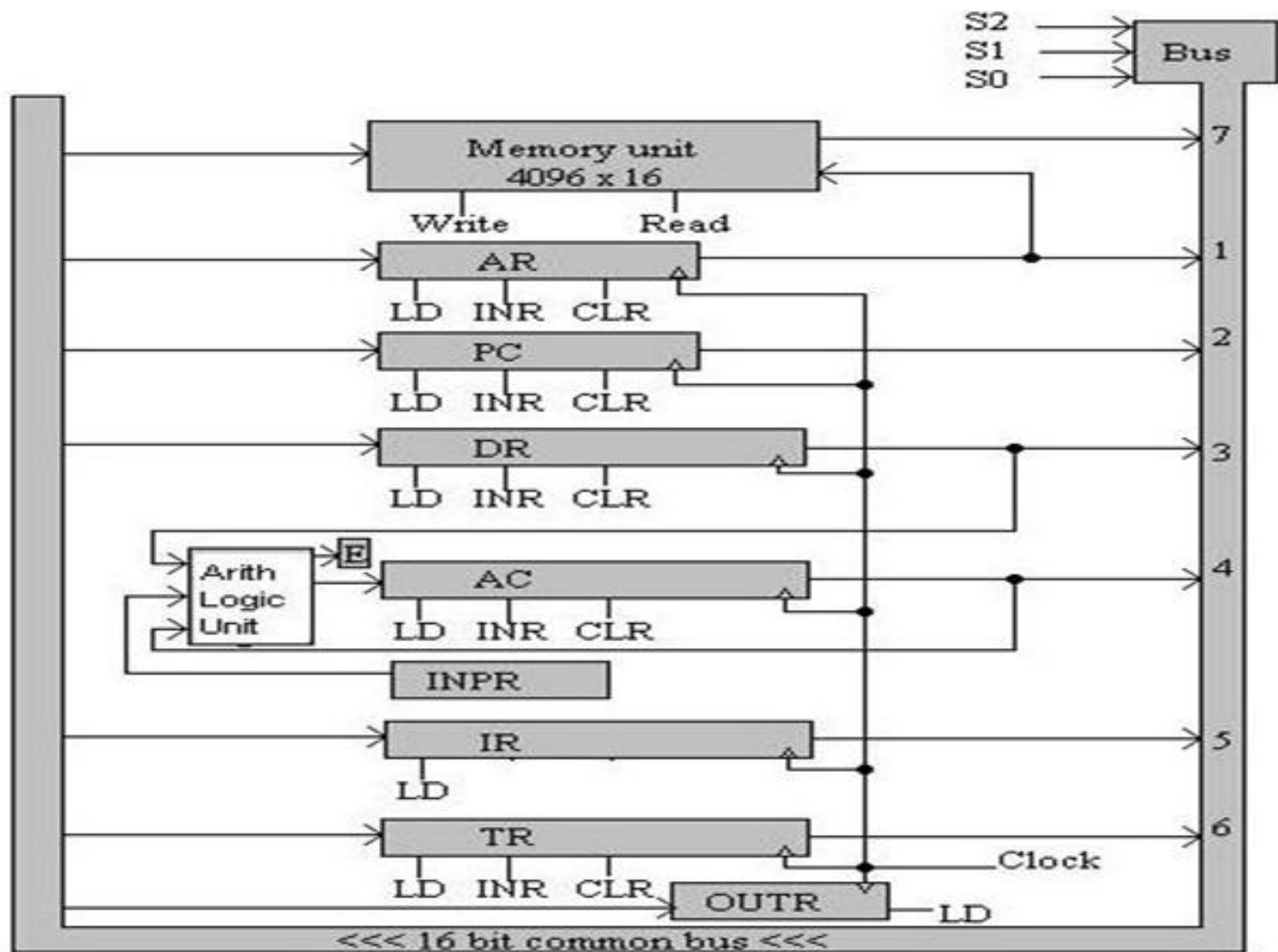
- Instructions are stored in consecutive memory locations and are executed sequentially one at a time.
- The control read an instruction from a specific address in memory and executes it: after that next instruction is read an executed, and so on.
- Register are needed for storing fetched instructions, and counters for computing the address of the next instruction.

Table: list of Registers for the basic computer:-

Register symbol	Number of bits	Register name	Function
<i>DR</i>	16	Data register	Holds memory operand
<i>AR</i>	12	Address register	Holds address for memory
<i>AC</i>	16	Accumulator	Processor register
<i>IR</i>	16	Instruction register	Holds instruction code
<i>PC</i>	12	Program counter	Holds address of instruction
<i>TR</i>	16	Temporary register	Holds temporary data
<i>INPR</i>	8	Input register	Holds input character
<i>OUTR</i>	8	Output register	Holds output character

Basic Computer register connected to a bus system

- The outputs of 8 registers and memory are connected to the common bus.
- Connections to bus lines are specified by selection lines S0, S1, and S2.
- A register load during the next clock pulse transition is selected with a LD (load) input.
- Memory write/read is enabled with write/read signals.

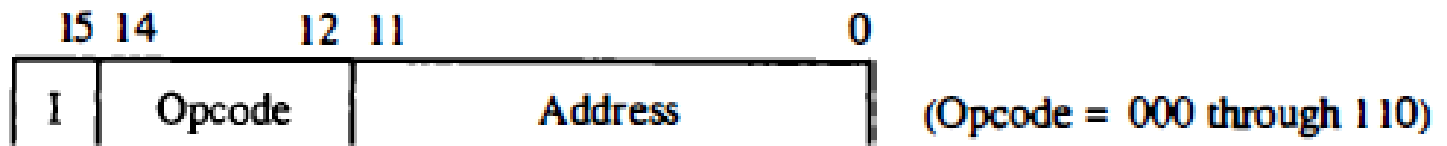


- INPR receives a character from an input device.
- OUTR receives a character from AC and delivers it to an output device.
- Bus receives data from 6 registers and the memory unit.
- 5 registers have three control lines: LD (load), INR (increment), and CLR (clear): equivalent to a binary counter with parallel load and synchronous clear.
- 2 registers have only a LD input.
- AR is used to specify memory address: no need for an address bus. 16 inputs to AC come from an adder and logic circuit with three sets of inputs: AC output, DR, INPR.

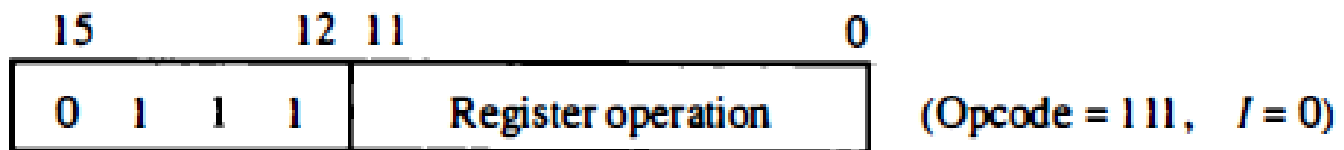
- Content of any register can be applied onto the bus, and an operation can be performed in the adder and logic circuit during the same clock cycle.
- The clock transition at the end of the cycle transfers the content of the bus into the designated register and the output of the adder and logic circuit into AC,
- e.g.: $DR \leftarrow AC$ and $AC \leftarrow DR$
- AC to the bus ($S_2S_1S_0 = 100$), enabling the LD of DR, transferring DR into AC (through adder and logic unit), and enabling LD of AC, all during the same clock cycle. The two transfers occur upon the arrival of the clock pulse transition at the end of the clock cycle.

The basic computer has three instruction code formats:

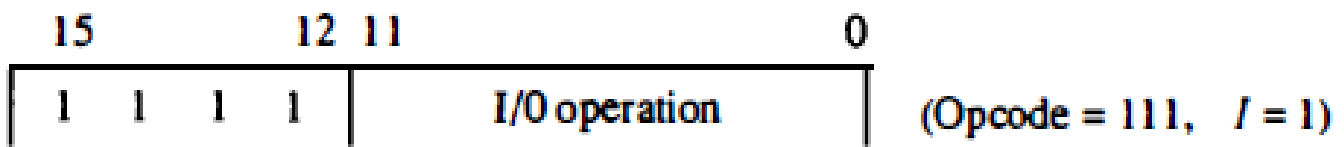
1. Register Reference instruction
2. Memory Reference instructions
3. Input –output instruction



(a) Memory – reference instruction



(b) Register – reference instruction



(c) Input – output instruction

- **A memory-reference instruction**
- Refers to memory address as an operand. the operand is always accumulator.
- Specifies 12 bit address, 3 bit opcode (other than 111) and 1 bit addressing mode for direct and indirect addressing.
- **Register reference instructions**
are recognized by the operation code 111 with a 0 in the leftmost bit (bit 15) of the instruction

- **Input –output Instruction:**
- An input-output instruction does not need a reference to memory and is recognized by the operation code 111 with a 1 in the leftmost bit of the instruction.
- The remaining 12 bits are used to specify the type of input-output operation or test performed

INSTRUCTION CYCLE:

- In the basic computer each instruction cycle consists of the following phases:
 - a) Fetch an instruction from memory.
 - b) Decode the instruction.
 - c) Read the effective address from memory if the instruction has an indirect address.

d) Execute the instruction

- After step d, the control goes back to step 1 to fetch, decode and execute the next instruction.
- This process continues unless a HALT instruction is encountered.

1. FETCH AN INSTRUCTION FROM THE MEMORY

- Initially PC is loaded with the address of the first instruction of the program.
- The sequence counter (SC) is cleared to 0. after each clock pulse, SC is incremented by one, so that the timing signals go through a sequence T_0, T_1, T_2 , and so on.

$$T_0: AR \leftarrow PC$$

$$T_1: IR \leftarrow M[AR], PC \leftarrow PC + 1$$

(2) Decode the Instruction:

The fetched instructions are decoded at time T₂, which can be specified with the microoperation.:-

$T_2 D_0 \dots D_7 \leftarrow \text{decode IR [12-14]},$

$AR \leftarrow \text{IR (0-11)},$

$I \leftarrow \text{IR(15)}$

(3) Read Effective Address:

- $D_7'I'T_3 : AR \leftarrow M[AR]$

Other instruction are identified with the specified control function and executed.

$D_7'I'T_3$: nothing

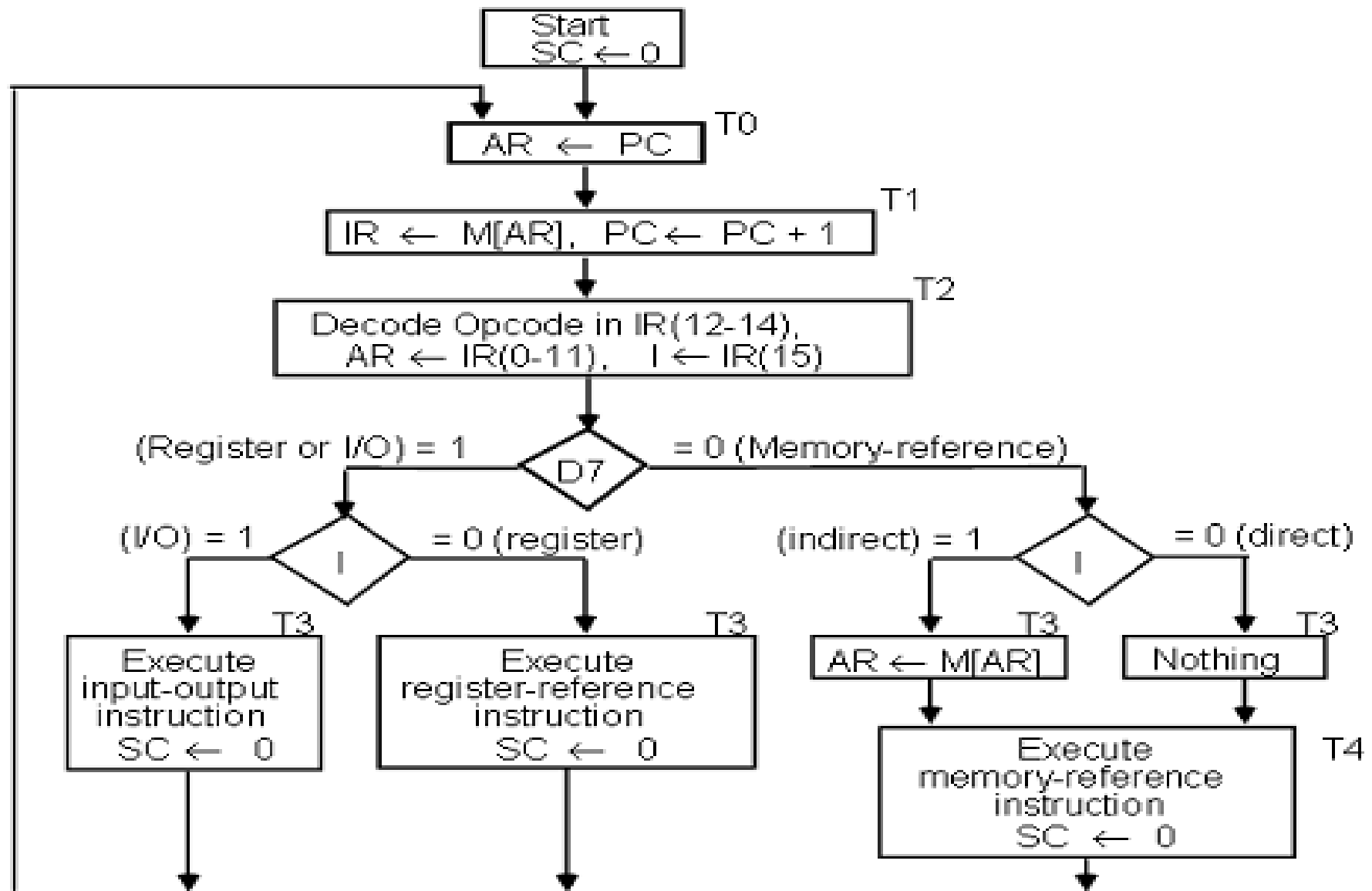
$D_7I'T_3$: execute register –reference instruction

D_7IT_3 : execute an input –output instruction

(4) Execute the instruction:

- After fetching the indirect address from the memory, instructions are executed .
- SC is initialized with 0 sign and control is passed to the first step.
- This process continues unless a HALT instruction is encountered.

Flowchart for instruction cycle (Fetch and Decode Phase)



Types of Instruction:

There are three types of instructions:

1. Memory Reference instructions
2. Register Reference instruction
3. Input –output instruction

(1) Memory Reference Instructions:

An instruction which has operation code D7=000 to 110 and i=1 or 0 then it is called memory – reference instructions.

MEMORY REFERENCE INSTRUCTIONS

Symbol	Operation Decoder	Symbolic Description
AND	D ₀	$AC \leftarrow AC \wedge M[AR]$
ADD	D ₁	$AC \leftarrow AC + M[AR], E \leftarrow C_{out}$
LDA	D ₂	$AC \leftarrow M[AR]$
STA	D ₃	$M[AR] \leftarrow AC$
BUN	D ₄	$PC \leftarrow AR$
BSA	D ₅	$M[AR] \leftarrow PC, PC \leftarrow AR + 1$
ISZ	D ₆	$M[AR] \leftarrow M[AR] + 1, \text{ if } M[AR] + 1 = 0 \text{ then } PC \leftarrow PC + 1$

(1)AND to AC :-

- This is an instruction that performs the AND logic operation on pairs of bits in AC and the memory word specified by the effective address.
- the result of the operation is transferred to AC. the micro operation that executes this instruction are:-

D0T4 : DR $\leftarrow$ M[AR]

D0T5:AC $\leftarrow$ AC AND Operation DR, SC $\leftarrow$ 0

(2) ADD to AC :-

- This instruction adds the contents of the memory word specified by the effectiveness address to the value of AC .
- the sum is transferred to AC and the output carry is transferred E flip flop.

D0T4: DR $\leftarrow$ M[AR]

D0T5: AC $\leftarrow$ AC + DR, E $\leftarrow$ Count, SC $\leftarrow$ 0

- **(3) LDA** : load to AC:- this instruction transfers the memory word specified by the effective address to AC. the micro operation needed to execute this instruction are:-

D2T4 : DR $\leftarrow$ M[AR]

D2T5 : AC $\leftarrow$ DR, SC $\leftarrow$ 0

- **(4) STA** : store AC:- this instruction stores the content of AC into the memory word specified by the effective address .

D3T4 : M[AR] $\leftarrow$ AC, SC $\leftarrow$ 0

(5) BUN:

- branch unconditionally :
- This instruction transfers the program to the instruction specified by the effective address.
- This instruction allows the programmer to specify an instruction out of sequence and we say that program branches unconditionally. the instruction is executed with one microoperation.

$D4T4:PC \leftarrow AR, SC \leftarrow 0$

(6) BSA:

- Branch and save return address: this instruction is useful for branching to a portion of the program called subroutine or procedure .
- when executed , the BSA instruction stores the address of next instruction in sequence into a memory location specified by the effective address.

D5T4: $M[AR] \leftarrow PC$, $AR \leftarrow AR+1$

D5T5 : $PC \leftarrow AR, SC \leftarrow 0$

(7) ISZ:

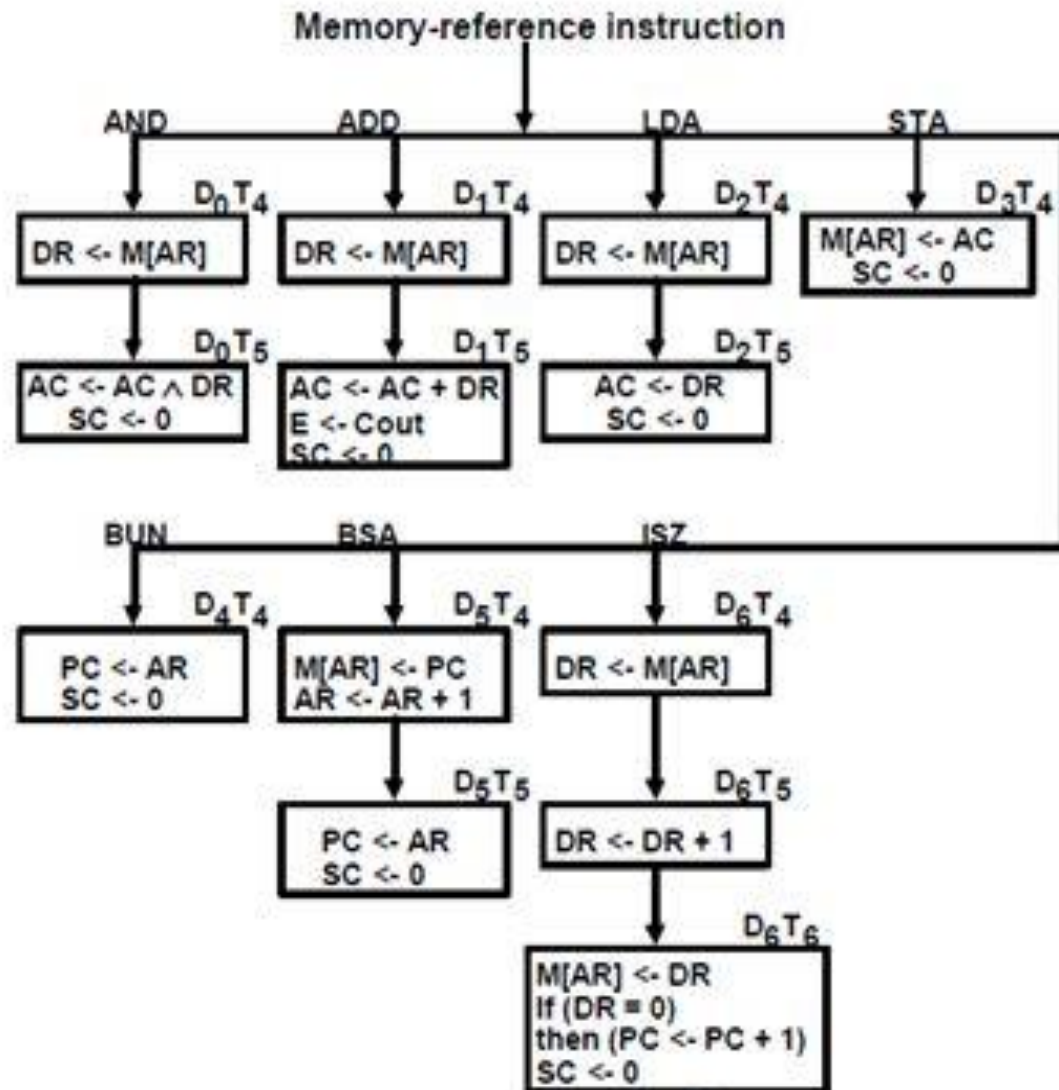
- Increment and skip if zero:-
- This instruction increments the word specified by the effective address and if the incremented value is equal to zero, PC is incremented by 1.

D6T4: $DR \leftarrow M[AR]$

D6T5: $DR \leftarrow DR + 1$

**D6T6 $M[AR] \leftarrow DR$, if $DR = 0$
then $PC \leftarrow PC + 1$, $SC \leftarrow 0$**

FLOWCHART FOR MEMORY REFERENCE INSTRUCTIONS



Register –Reference Instruction:-

- Instruction which has operation code D7=111 and i=0 then it is called as register –reference instruction
- The first seven register-reference instructions perform clear, complement, circular shift, and increment microoperation in the AC or E registers.
- The next four register cause a skip of the next instruction in sequence when a stated condition is satisfied.

	r:	SC $\leftarrow$ 0
CLA	rB ₁₁ :	AC $\leftarrow$ 0
CLE	rB ₁₀ :	E $\leftarrow$ 0
CMA	rB ₉ :	AC $\leftarrow$ AC'
CME	rB ₈ :	E $\leftarrow$ E'
CIR	rB ₇ :	AC $\leftarrow$ shr AC, AC(15) $\leftarrow$ E, E $\leftarrow$ AC(0)
CIL	rB ₆ :	AC $\leftarrow$ shl AC, AC(0) $\leftarrow$ E, E $\leftarrow$ AC(15)
INC	rB ₅ :	AC $\leftarrow$ AC + 1
SPA	rB ₄ :	if (AC(15) = 0) then (PC $\leftarrow$ PC+1)
SNA	rB ₃ :	if (AC(15) = 1) then (PC $\leftarrow$ PC+1)
SZA	rB ₂ :	if (AC = 0) then (PC $\leftarrow$ PC+1)
SZE	rB ₁ :	if (E = 0) then (PC $\leftarrow$ PC+1)
HLT	rB ₀ :	S $\leftarrow$ 0 (S is a start-stop flip-flop)

- **(3) Input –output Instruction :-** input-output instructions have an operation code 1111 and are recognized by the control when D7=1 and I=1

INPUT-OUTPUT INSTRUCTIONS

$D_7IT_3 = p$
 $IR(i) = B_i, i = 6, \dots, 11$

INP	$pB_{11}:$	$AC(0-7) \leftarrow INPR, FGI \leftarrow 0$	Input char. to AC
OUT	$pB_{10}:$	$OUTR \leftarrow AC(0-7), FGO \leftarrow 0$	Output char. from AC
SKI	$pB_9:$	if($FGI = 1$) then ($PC \leftarrow PC + 1$)	Skip on input flag
SKO	$pB_8:$	if($FGO = 1$) then ($PC \leftarrow PC + 1$)	Skip on output flag
ION	$pB_7:$	$IEN \leftarrow 1$	Interrupt enable on
IOF	$pB_6:$	$IEN \leftarrow 0$	Interrupt enable off

Timing and control :

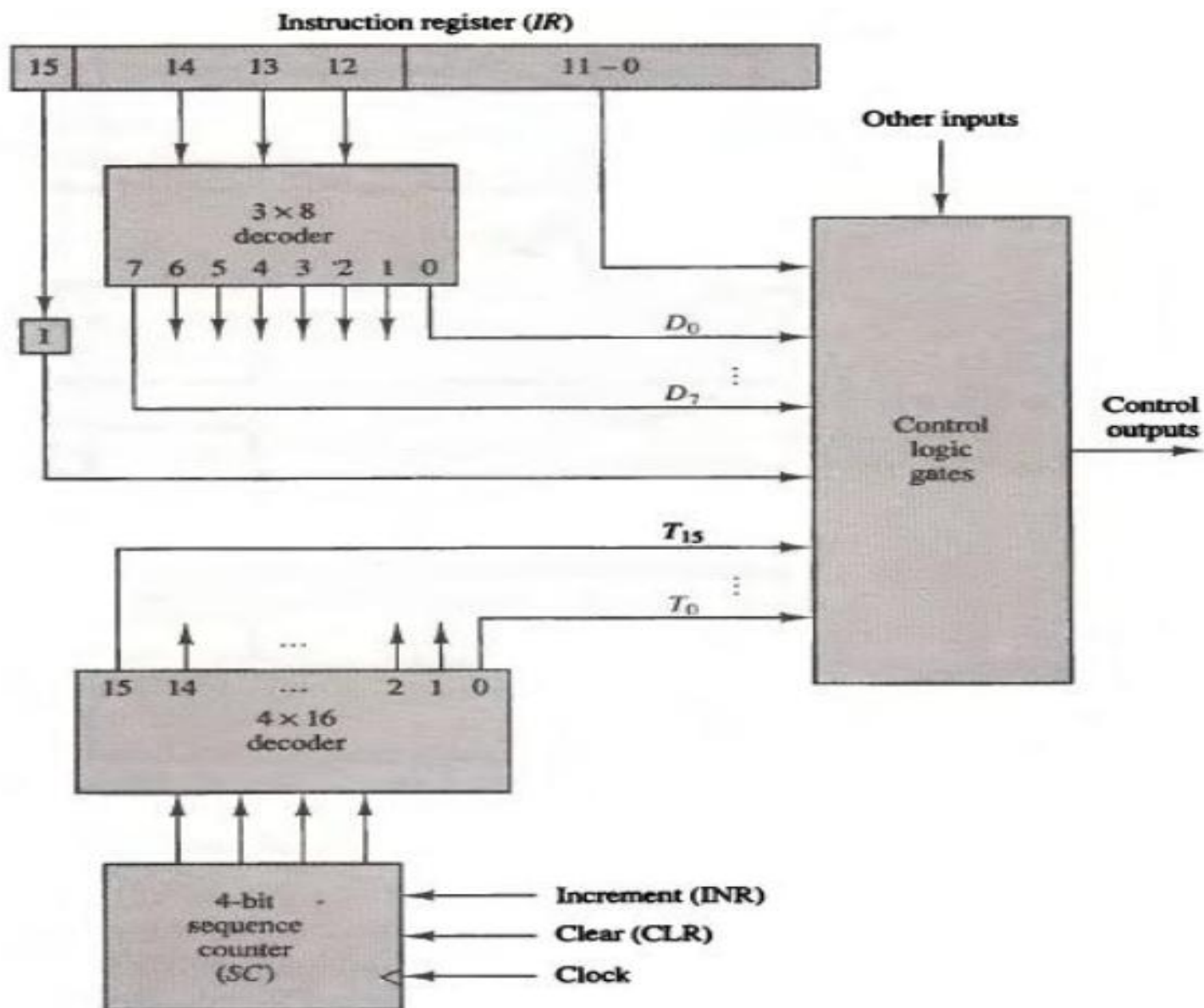
- The timing for all register of computer is controlled by a master clock generator.
- Clock pulse applied to all FF, registers in the system
- Clock pulse do not change the state of the register unless the registers is enabled by a control signal.
- **There are two types of control organization:**
 - 1. Hardwired Control**
 - 2. Micro-Programmed Control**

Hardwired control:

- The control logic is implemented with gates, flip-flops, decoders and other digital circuits.
- It requires changes in the wiring among the various components.

Construction and working:

- The control unit consists of two decoders, a sequence counter and a number of control logic gates.
- A read instruction is placed in the instruction register(IR).
- In this instruction register, there are 3 parts-1 I bit, operation code, and bits 0 to 11. I bit is transferred to flip –flop ,the operation code in bits 12 through 14 are decode in D_0 - D_7 by 3:8 decoder, bits 0 to 11 is applied to control logic gates.
- Next counter decoder 4:16 provided 16 timing signals t_0 through t_{15} .
- The sequence counter(SC) can be incremented or cleared synchronously.

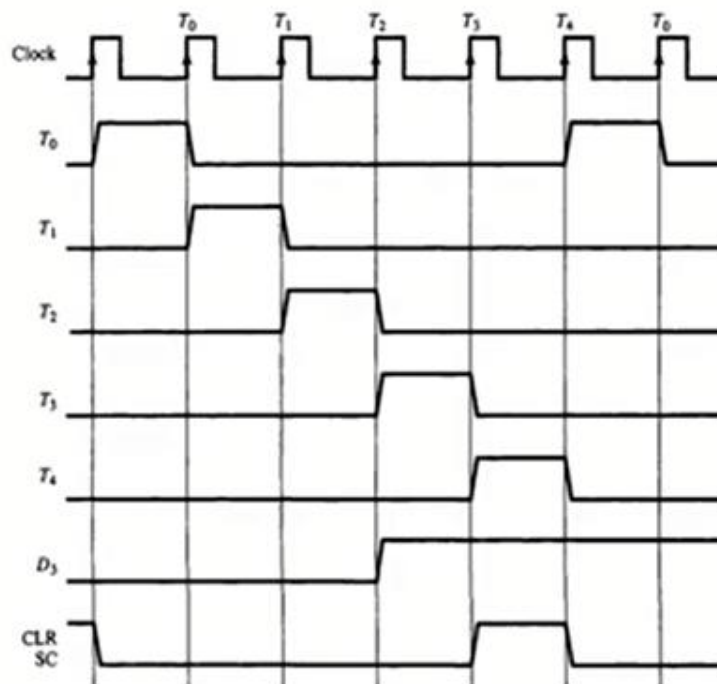


Timing Signals

- The sequence counter SC can be incremented or cleared synchronously.
- The counter is incremented to provide the sequence of timing signals out of the 4*16 decoder. (it happens most of the time)
- The counter is cleared to 0, causing the next active timing signal to be T0. (once in a while)
- Example- SC is incremented to provide timing signals T0,T1,T2,T3 and T4 in sequence. At time T4, SC is cleared to 0 if decoder output D3 is active. This is expressed symbolically by the statement

$$D_3T_4: SC \leftarrow 0$$

Example of control timing signal



- The SC responds to the positive transition of the clock.
- Initially, CLR input of SC is active
- The first positive transition of the clock clears SC to 0, which in turn activates the timing signal T₀ out of the decoder.
- The positive clock transition labelled T₀ in the diagram will trigger only those registers whose control inputs are connected to timing signal T₀.
- SC is incremented with every positive clock transition unless its clear input is active. This produces the sequence of timing signals T₀, T₁, T₂, T₃, T₄ and so on.
- If SC is not cleared, the timing signals will continue with T₅, T₆ up to T₁₅ and back to T₀.
- Output D₃ from the operation decoder becomes active at the end of timing signal T₂.
- When timing signal T₄ becomes active the output of the AND gate that implements the control function D₃T₄ becomes active.
- This signal is applied to the CLR input of SC.
- On the next positive clock transition the counter is cleared to 0, this causes the T₀ to become active instead of T₅.

- **Microoperations:**

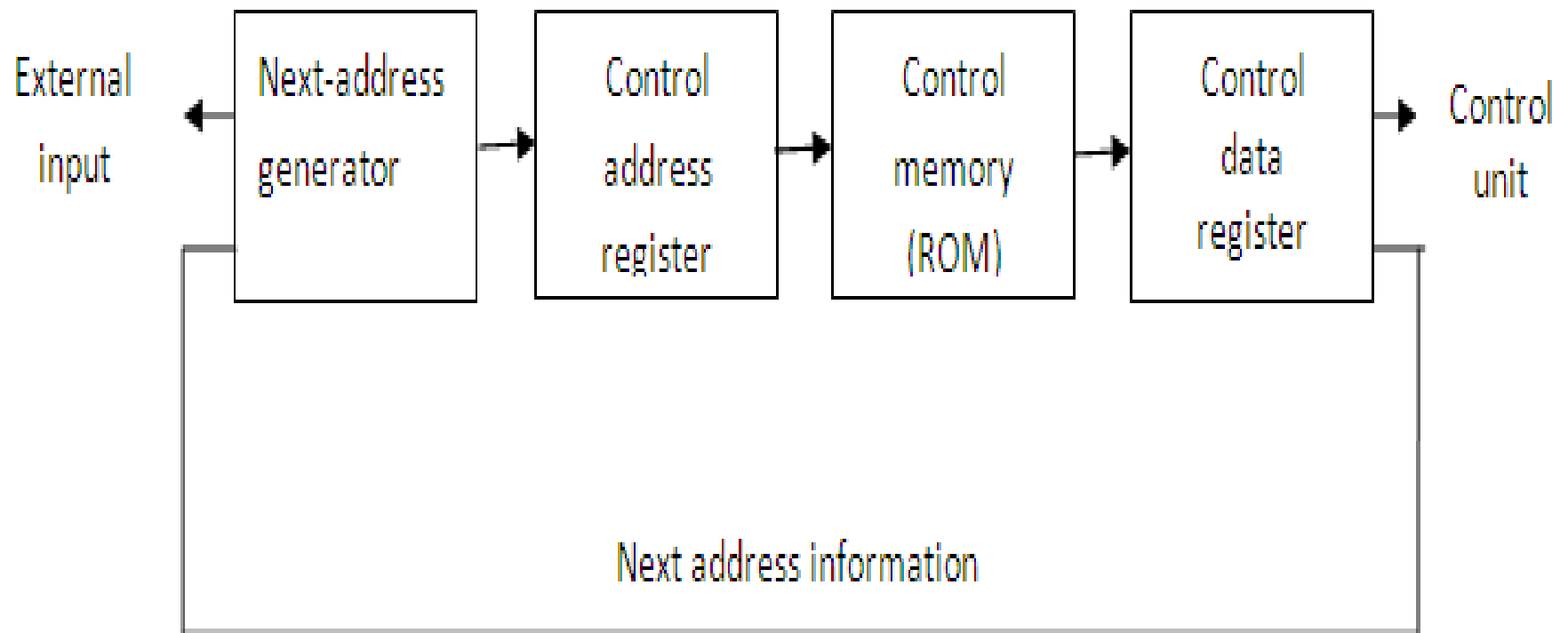
- In computer central processing units, micro-operations are detailed low-level instructions used in some designs to implement complex machine instructions (sometimes termed macro-instructions in this context).

- **Micro instruction:**

- A symbolic microprogram can be translated into its binary equivalent by means of an assembler.
- Each line of the assembly language microprogram defines a symbolic microinstruction.
- Each symbolic microinstruction is divided into five fields: label, microoperations, CD, BR, and AD.

Micro-programmed control:

- Control information is stored in a control memory. control memory is programmed to initiate the required sequence of micro operations.
- Any modification and changes can be done by updating the microprogram in control memory.



- While the microoperations are being executed, the next address is computed in the next address generator circuit and then transferred into the control address register to read the next microinstruction.
- Thus a microinstruction contains bits for initiating microoperations in the data processor part and bits that determine the address sequence for the control memory.
- The next address generator is sometimes called a ***micro-program sequencer***, as it determines the address sequence that is read from control memory.
- Typical functions of a micro-program sequencer are incrementing the control address register by one, loading into the control address register an address from control memory, transferring an external address, or loading an initial address to start the control operations.

- The control data register holds the present microinstruction while the next address is computed and read from memory.
- The data register is sometimes called a *pipeline register*.
- It allows the execution of the microoperations specified by the control word simultaneously with the generation of the next microinstruction.
- This configuration requires a two-phase clock, with one clock applied to the address register and the other to the data register.
- The main advantage of the micro programmed control is the fact that once the hardware configuration is established; there should be no need for further hardware or wiring changes.
- If we want to establish a different control sequence for the system, all we need to do is specify a different set of microinstructions for control memory.

Interrupt

- An interrupt is a signal to the processor emitted by hardware or software indicating an event that needs immediate attention.
- An interrupt alerts the processor to a high-priority condition requiring the interruption of the current code the processor is executing.
- The processor responds by suspending its current activities, saving its state, and executing a function called an *interrupt handler* (or an interrupt service routine, ISR) to deal with the event.

- This interruption is temporary, and, after the interrupt handler finishes, the processor resumes normal activities.
- **Interrupt are of two types**
 - (1) Hardware Interrupts**
 - (2) Software interrupt**

Hardware Interrupts

- are used by devices to communicate that they require attention from the operating system.
- hardware interrupts are implemented using electronic alerting signals that are sent to the processor from an external device,
- which is either a part of the computer itself, such as a disk controller, or an external peripheral.
- For example, pressing a key on the keyboard or moving the mouse triggers hardware interrupts that cause the processor to read the keystroke or mouse position.

External Interrupts

- External interrupts come from input-output (I/O) devices, from a timing device, from a circuit monitoring the power supply, or from any other external source.
- Examples that cause external interrupts are I/O device requesting transfer of data, I/O device finished transfer of data, elapsed time of an event, or power failure.
- External Interrupts are asynchronous.
- External Interrupts depends on external conditions that are independent of the program being executed at the time.

Two types of Hardware Interrupts

- (1) Maskable Interrupt: The hardware interrupts which can be delayed when a much high priority interrupt has occurred at the same time.
- (2) Non Maskable Interrupt: The hardware interrupts which can not be delayed and should processed by the processor immediately.

Software interrupts:

- A software interrupt is a special call instruction that behaves like an interrupt rather than a subroutine call.
- It can be used by the programmer to initiate an interrupt procedure at any desired point in the program.
- This instruction provides means for switching from a CPU user mode to the supervisor mode.
- Certain operations in the computer may be assigned to the supervisor mode only, as for example, a complex input or output transfer procedure

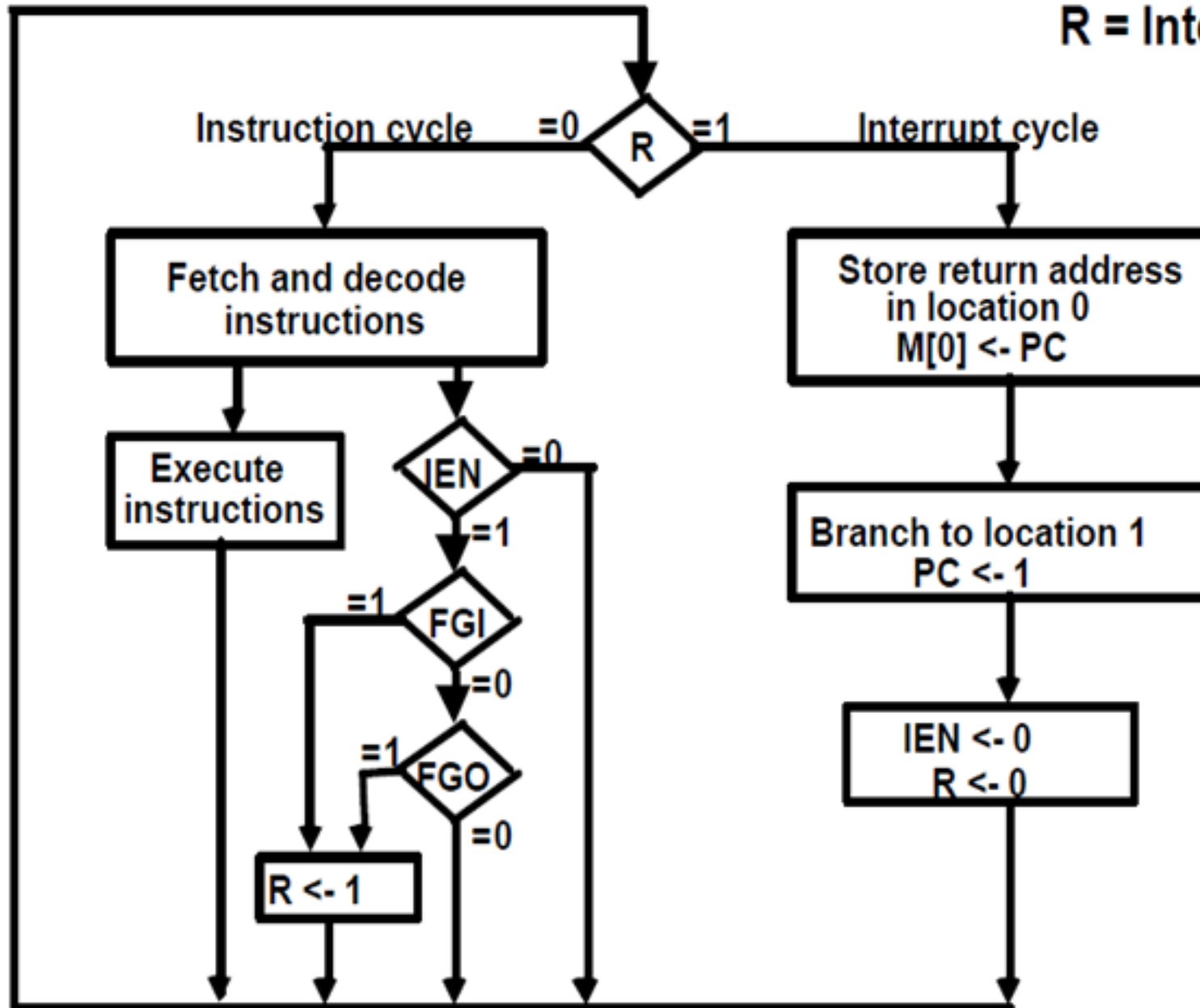
Internal Interrupts:

- Internal interrupts arise from illegal use of an instruction or data. Internal interrupts are also called traps.
- Examples of interrupts caused by internal error conditions are register overflow, attempt to divide by zero, an invalid operation code, stack overflow, and protection violation.
- These error conditions usually occur as a result of a premature termination of the instruction execution.
- Internal interrupts are synchronous with the program. . If the program is rerun, the internal interrupts will occur in the same place each time.

- **Interrupt cycle:**
- The interrupt cycle is added to the instruction cycle.
- Processor checks for interrupt indicated by an interrupt flag.
- If there is NO interrupt → Fetch next instruction

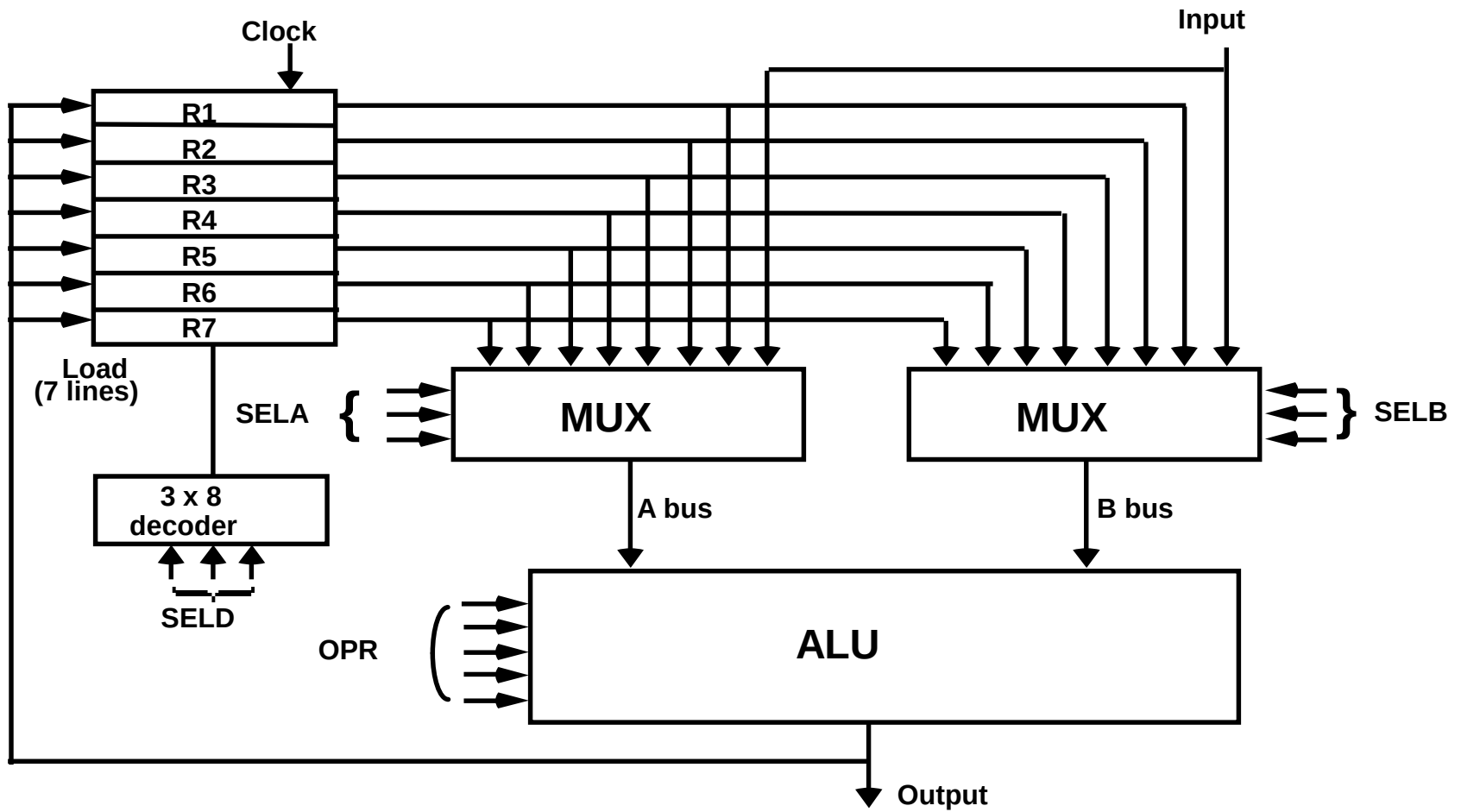
- If there is an interrupt:
 - a. Suspend operation of the program
 - b. Save its context
 - c. Set PC to start address of the interrupt handler
 - d. Process the interrupt
 - e. Restore the context of the original program and continue its execution.

R = Interrupt f/f



GENERAL REGISTER ORGANIZATION

- Generally CPU has seven general registers.
- Register organization show how registers are selected and how data flow between register and ALU.
- A decoder is used to select a particular register.
- The output of each register is connected to two multiplexers to form the two buses A and B.
- The selection lines in each multiplexer select the input data for the particular bus.



- The A and B buses form the two inputs of an ALU.
- The operation select lines decide the micro operation to be performed by ALU.
- The result of the micro operation is available at the output bus.
- The output bus connected to the inputs of all registers, thus by selecting a destination register it is possible to store the result in it.

OPERATION OF CONTROL UNIT

The control unit directs the information flow through ALU by:

- Selecting various *Components* in the system
- Selecting the *Function* of ALU

Example: $R1 \leftarrow R2 + R3$

[1] MUX A selector (SELA): $BUS\ A \leftarrow R2$

[2] MUX B selector (SELB): $BUS\ B \leftarrow R3$

[3] ALU operation selector (OPR): ALU to ADD

[4] Decoder destination selector (SELD): $R1 \leftarrow Out\ Bus$

Control Word

SELA	SELB	SELD	OPR
------	------	------	-----

Encoding of register selection fields

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

ALU CONTROL

Encoding of ALU operations

OPR Select	Operation	Symbol
00000	Transfer A	TSFA
00001	Increment A	INCA
00010	ADD A + B	ADD
00101	Subtract A - B	SUB
00110	Decrement A	DECA
01000	AND A and B	AND
01010	OR A and B	OR
01100	XOR A and B	XOR
01110	Complement A	COMA
10000	Shift right A	SHRA
11000	Shift left A	SHLA

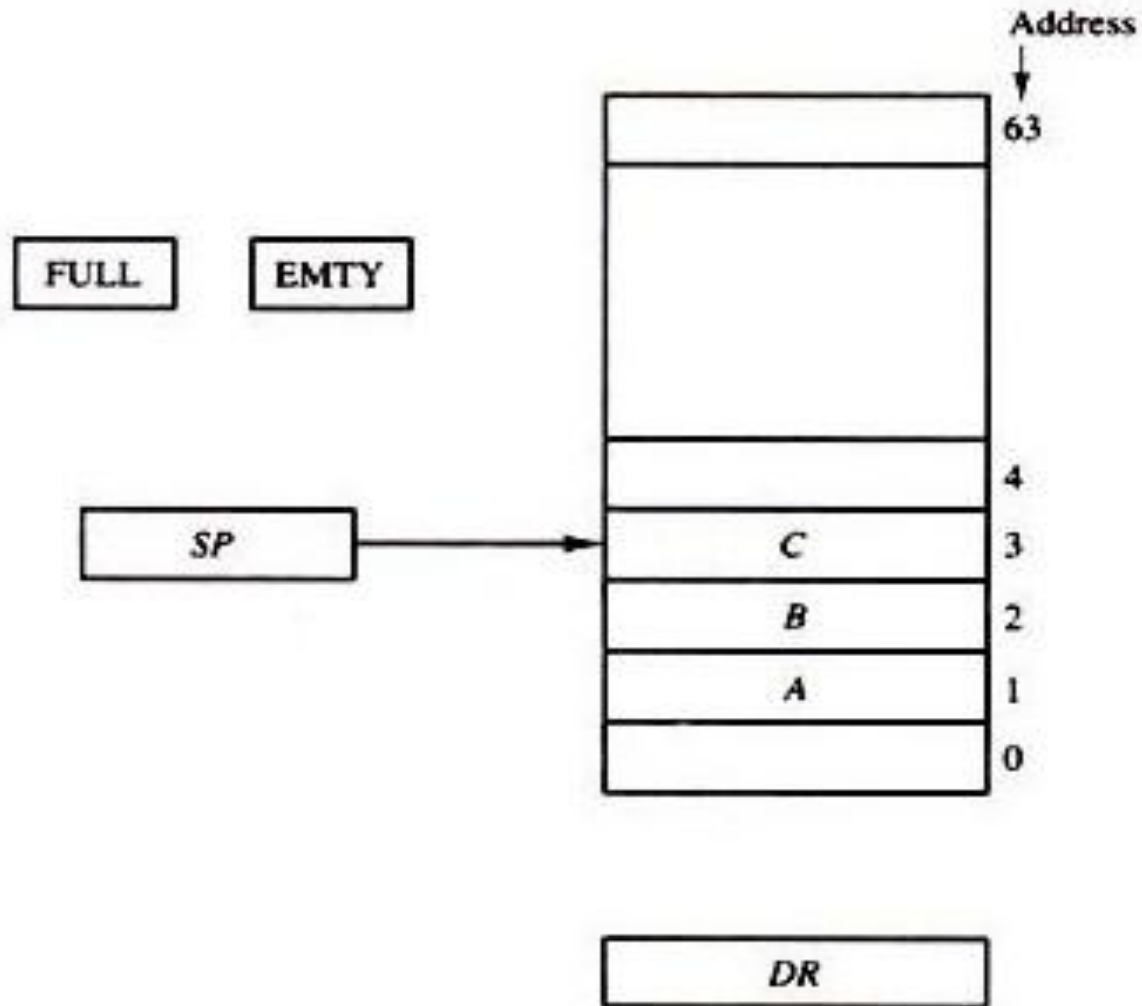
Examples of ALU Microoperations

Microoperation	Symbolic Designation				Control Word
	SELA	SELB	SELD	OPR	
$R1 \leftarrow R2 - R3$	R2		R3	R1 SUB	010 011 001 00101
$R4 \leftarrow R4 \vee R5$	R4		R5	R4 OR	100 101 100 01010
$R6 \leftarrow R6 + 1$	R6	-	R6	INCA	110 000 110 00001
$R7 \leftarrow R1$	R1	-	R7	TSFA	001 000 111 00000
$\text{Output} \leftarrow R2$	R2		-	None TSFA	010 000 000 00000
$\text{Output} \leftarrow \text{Input}$	Input		-	None TSFA	000 000 000 00000
$R4 \leftarrow \text{shl } R4$	R4	-	R4	SHLA	100 000 100 11000
$R5 \leftarrow 0$	R5	R5	R5	XOR	101 101 101 01100

Stack Organization

- A stack is a storage device that stores information in such a manner that the item stored last is the first item retrieved(LIFO).
- The register that holds the address for the stack is called a stack pointer (SP) because its value always points at the top item in the stack.
- There are two types of stack organization
 - 1.Register Stack-Built using registers
 - 2.Memory Stack-Logical part of memory allocated as stack

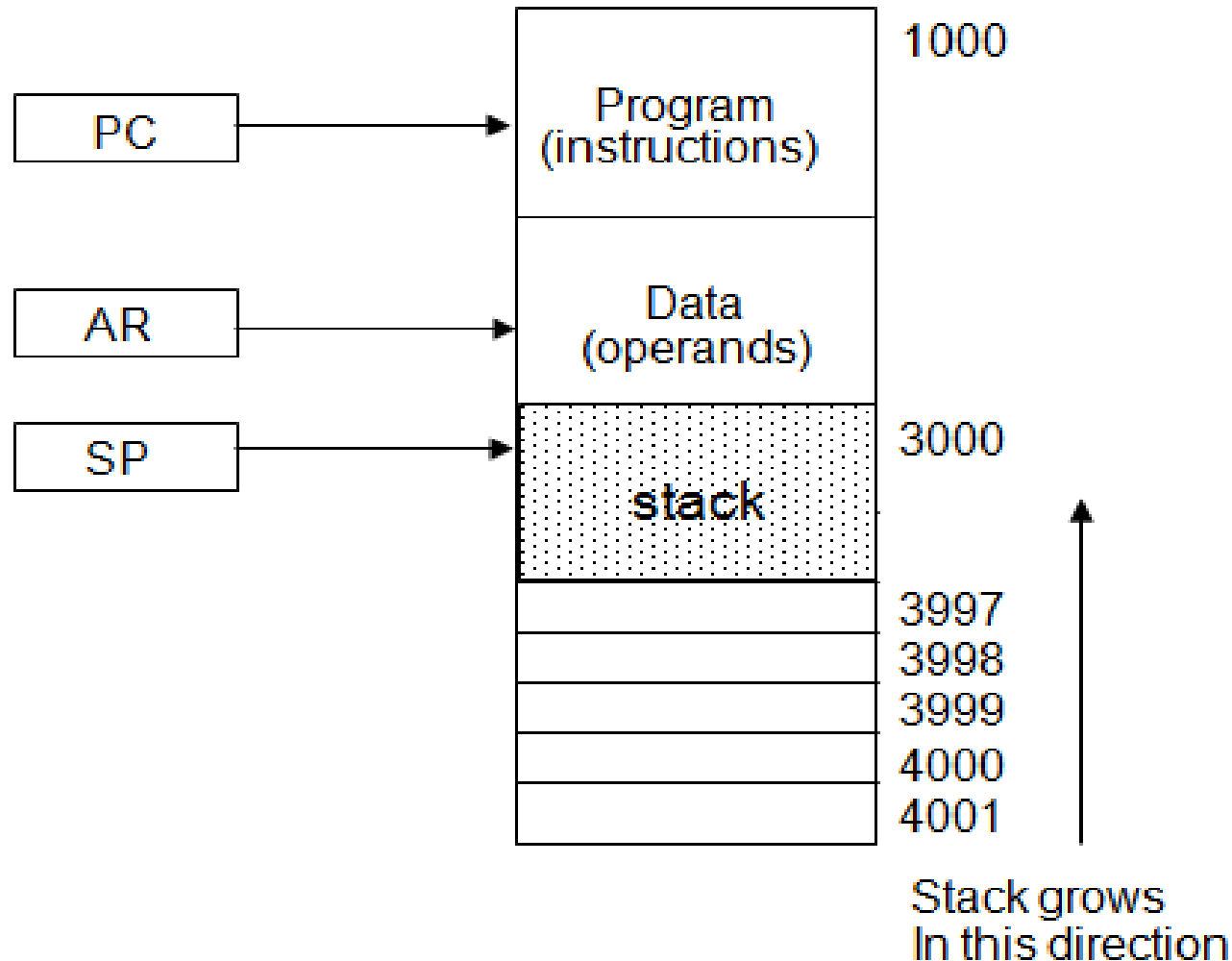
Register Stack



- ***PUSH:***
 - If the stack is not full ($\text{FULL} = 0$), a new item is inserted with a push operation. The push operation consists of the following sequences of microoperations:
- $\text{SP} \leftarrow \text{SP} + 1$ Increment stack pointer
- $\text{M}[\text{SP}] \leftarrow \text{DR}$ Write item on top of the stack
- IF ($\text{SP} = 0$) then ($\text{FULL} \leftarrow 1$) Check if stack is full
- $\text{EMPTY} \leftarrow 0$ Mark the stack not empty

- ***POP:***
 - A new item is deleted from the stack if the stack is not empty (if $EMPTY = 0$). The pop operation consists of the following sequences of microoperations:
- $DR \leftarrow M[SP]$ Read item on top of the stack
- $SP \leftarrow SP - 1$ Decrement stack pointer
- IF $(SP = 0)$ then $(EMPTY \leftarrow 1)$ Check if stack is empty
- $FULL \leftarrow 0$ Mark the stack not full

Memory Stack



- ***PUSH***
 - A new item is inserted with the push operation as follows:
- $SP \leftarrow SP - 1$
- $M[SP] \leftarrow DR$
 - The stack pointer is decremented so that it points at the address of the next word.
 - A memory write operation inserts the word from DR into the top of the stack.
-

- ***POP***
 - A new item is deleted with a pop operation as follows:
- $DR \leftarrow M[SP]$
- $SP \leftarrow SP + 1$
 - The top item is read from the stack into DR.
 - The stack pointer is then incremented to point at the next item in the stack.
 - The two microoperations needed for either the push or pop are (1) an access to memory through SP, and (2) updating SP.
 - Which of the two microoperations is done first and whether SP is updated by incrementing or decrementing depends on the organization of the stack.

Reverse Polish Notation(RPN)

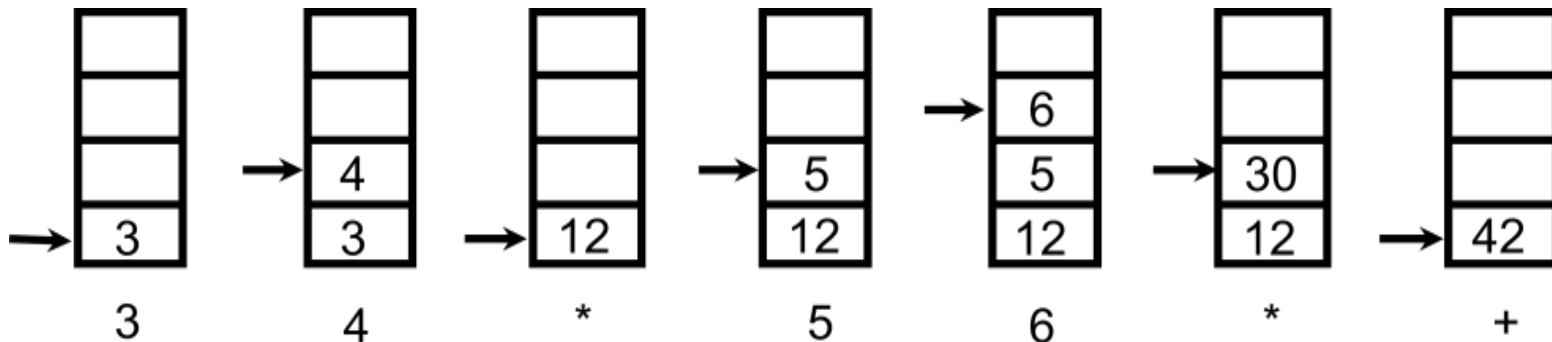
- The postfix RPN notation, referred to as Reverse Polish Notation (RPN), places the operator after the operands.
- The following examples demonstrate the three representations:
 - $A + B$ Infix notation
 - $+ A B$ Prefix or Polish notation
 - $A B +$ Postfix or reverse Polish notation
- The reverse Polish notation is in a form suitable for stack manipulation. The expression

- $A * B + C * D$ is written in reverse Polish notation as
- $A B * C D * +$
 - The conversion from infix notation to reverse Polish notation must take into consideration the operational hierarchy adopted for infix notation.
 - This hierarchy dictates that we first perform all arithmetic inside inner parentheses, then inside outer parentheses, and do multiplication and division operations before addition and subtraction operations.

- ***Evaluation of Arithmetic Expressions***

- Any arithmetic expression can be expressed in parenthesis-free Polish notation, including reverse Polish notation

- $(3 * 4) + (5 * 6) = 3 \ 4 \ * \ 5 \ 6 \ * \ +$



Instruction formats

- Three Address Instructions
- Two Address Instructions
- One Address Instructions
- Zero Address Instructions

- **Three Address Instructions:**

- Computers with three-address instruction formats can use each address field to specify either a processor register or a memory operand.
- The program in assembly language that evaluates $X = (A + B) * (C + D)$ is shown below.

- `ADD R1, A, B` $R1 \leftarrow M[A] + M[B]$
- `ADD R2, C, D` $R2 \leftarrow M[C] + M[D]$
- `MUL X, R1, R2` $M[X] \leftarrow R1 * R2$

- **Two Address Instructions:**

- Two address instructions are the most common in commercial computers. Here again each address field can specify either a processor register or a memory word.
- The program to evaluate $X = (A + B) * (C + D)$ is as follows:

- | | |
|--------------|------------------|
| • MOV R1, A | R1 <- M [A] |
| • ADD R1, B | R1 <- R1 + M [B] |
| • MOV R2, C | R2 <- M [C] |
| • ADD R2, D | R2 <- R2 + M [D] |
| • MUL R1, R2 | R1 <- R1 * R2 |
| • MOV X, R1 | M [X] <- R1 |

One Address Instructions:

- One address instructions use an implied accumulator (AC) register for all data manipulation. For multiplication and division there is a need for a second register.
- However, here we will neglect the second register and assume that the AC contains the result of all operations. The program to evaluate $X = (A + B) * (C + D)$ is

•	LOAD	A	AC <- M [A]
•	ADD	B	AC <- AC + M [B]
•	STORE	T	M [T] <- AC
•	LOAD	C	AC <- M [C]
•	ADD	D	AC <- AC + M [D]
•	MUL	T	AC <- AC * M [T]
•	STORE	X	M [X] <- AC

Zero Address Instructions:

- A stack-organized computer does not use an address field for the instructions ADD and MUL.
- The PUSH and POP instructions, however, need an address field to specify the operand that communicates with the stack.
- The program to evaluate $X = (A + B) * (C + D)$ will be written for a stack-organized computer.
- To evaluate arithmetic expressions in a stack computer, it is necessary to convert the expression into reverse polish notation.

▪ PUSH	A	TOS <- A
▪ PUSH	B	TOS <- B
▪ ADD		TOS <- (A + B)
▪ PUSH	C	TOS <- B
▪ PUSH	D	TOS <- D
▪ ADD		TOS <- (C + D)
▪ MUL		TOS <- (C + D) * (A + B)
▪ POP	X	M [X] <- TOS

Addressing Modes

- The techniques for specifying the address of the operands are known as addressing modes.
- The address of an operand is known as effective address.
- Types of addressing modes

- **Implied mode:**

- In this mode the operands are specified implicitly in the definition of the instruction.
- For example, the instruction “complement accumulator” is an implied-mode because the operand in the accumulator register is implied in the definition of the instruction.
- Example:-CMA: Take complement of the content of AC

- **Immediate Mode:**

- In this mode the operand is specified in the instruction itself. In other words, an immediate-mode instruction has an operand field rather than an address field.
- The operand field contains the actual operand to be used in conjunction with the operation specified in the instruction.
- Immediate mode of instructions is useful for initializing register to constant value.
- Example: ADD 05

- **Register Mode:**

- In this mode the operands are in registers that within the CPU. The particular register is selected from a register field in the instruction.
- Example: MOV R1,R2

-

- **Register Indirect Mode:**

- In this mode the instruction specifies a register in the CPU whose contents give the address of the operand in memory.
- The advantage of this mode is that address field of the instruction uses fewer bits to select a register than would have been required to specify a memory address directly.
- Example: ADD R1, (R2)

- **Direct Address Mode:**

- In this mode the effective address is equal to the address part of the instruction.
- The operand resides in memory and its address is given directly by the address field of the instruction.
- Example: LD ADR AC <- M[ADR]

- **Indirect Address Mode:**

- In this mode the address field of the instruction gives the address where the effective address is stored in memory.
- Control fetches the instruction from memory

- and uses its address part to access memory again to read the effective address.
- The effective address in this mode is obtained from the following computational:
- Effective address = address part of instruction + content of CPU register
- Example: $LD@ADR \ AC \leftarrow M[M[ADR]]$

- **Relative Address Mode:**

- In this mode the content of the program counter is added to the address of the instruction in order to obtain the effective address.
- When this number is added to the content of the program counter, the result produces an effective address whose position in memory is relative to the address of the next instruction.

Example: LD\$ ADR AC <- M[PC+ADR]

- **Indexed Addressing Mode:**

- In this mode the content of an index register is added to the address part of the instruction to obtain the effective address.
- The indexed register is a special CPU register that contain an index value.

– Example:-LD ADR (X) AC <- M[ADR +XR]

- **Autoincrement or Autodecrement Mode:**

- This is similar to the register indirect mode expect that the register is incremented or decremented after (or before) its value is used to access memory.
- When the address stored in the register refers to a table of data in memory, it is necessary to increment or decrement the register after every access to the table. This can be achieved by using the increment or decrement instruction.
- Example of Auto increment : Add R1, (R2)+
- Example of Auto decrement: Add R1, -(R2)

RISC

- It stands for Reduced Instruction Set Computer.
- It is a type of microprocessor architecture that uses a small set of instructions of uniform length.
- These are simple instructions which are generally executed in one clock cycle.
- RISC chips are relatively simple to design and inexpensive.
- The setback of this design is that the computer has to repeatedly perform simple operations to execute a larger program having a large number of processing operations.

-

Examples: SPARC, POWER PC etc.

CISC

- It stands for Complex Instruction Set Computer.
- These processors offer the users, hundreds of instructions of variable sizes.
- CISC architecture includes a complete set of special purpose circuits that carry out these instructions at a very high speed.
- These instructions interact with memory by using complex addressing modes.
- CISC processors reduce the program size and hence lesser number of memory cycles are required to execute the programs. This increases the overall speed of execution.
- **Examples:** Intel architecture, AMD

RISC vs. CISC

CISC	RISC
Emphasis on hardware	Emphasis on software
Multiple instruction sizes and formats	Instructions of same set with few formats
Less registers	Uses more registers
More addressing modes	Fewer addressing modes
Extensive use of microprogramming	Complexity in compiler
Instructions take a varying amount of cycle time	Instructions take one cycle time
Pipelining is difficult	Pipelining is easy